HSLU Hochschule Luzern

Time Series Analysis

Deep Learning for Time Series

Dr. Ludovic Amruthalingam
ludovic.amruthalingam@hslu.ch

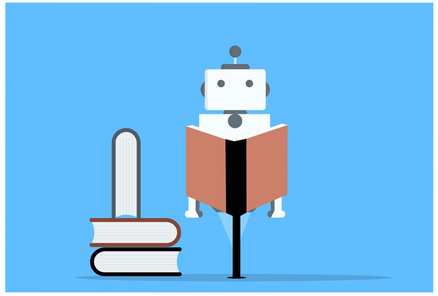
Informatik

FH Zentralschweiz

Outline

- Deep learning
- Feed forward neural networks
- Activation functions
- Loss functions
- Gradient descent
- Convolutional neural networks
- Recurrent neural networks
- Generalized attention model
- Transformers
- Practical tips

Recap – Machine learning



Machine learning studies **algorithms** that autonomously "learn" to perform **tasks** from **data** X.

$$t : X \rightarrow Y$$

Instead of manually defining specific steps and instructions, these algorithms are designed to automatically **extract patterns and statistical relationships** from data that are useful to complete the task.

$$m_\theta : X \rightarrow Y$$

An algorithm is considered to "learn" if its **performance** improves as it processes data. The training procedure optimizes the parameters θ of the algorithm's model m_θ with respect to a performance measure P:

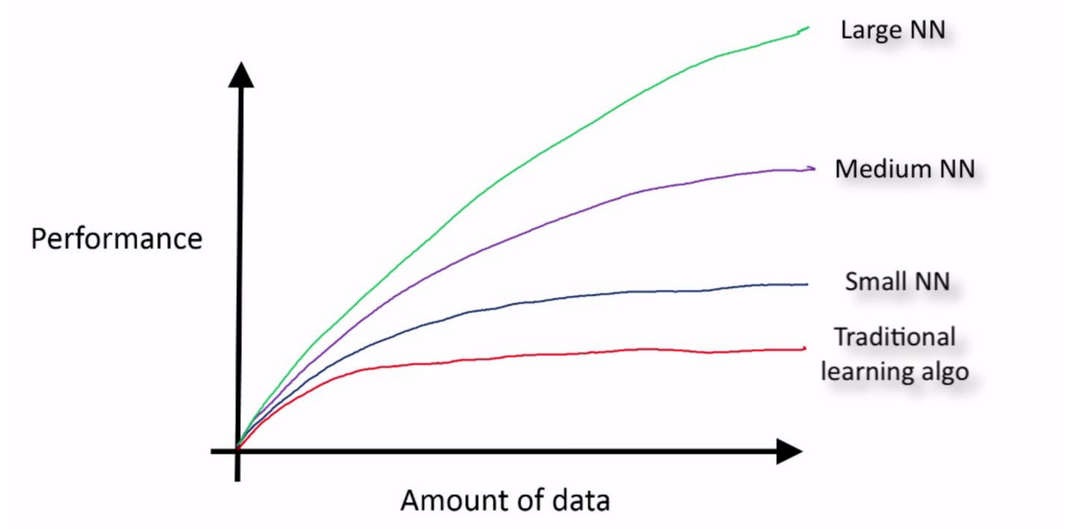
$$\theta^* = \max_\theta \sum_{\forall x \in X} P(t(x), m_\theta(x))$$

Deep learning

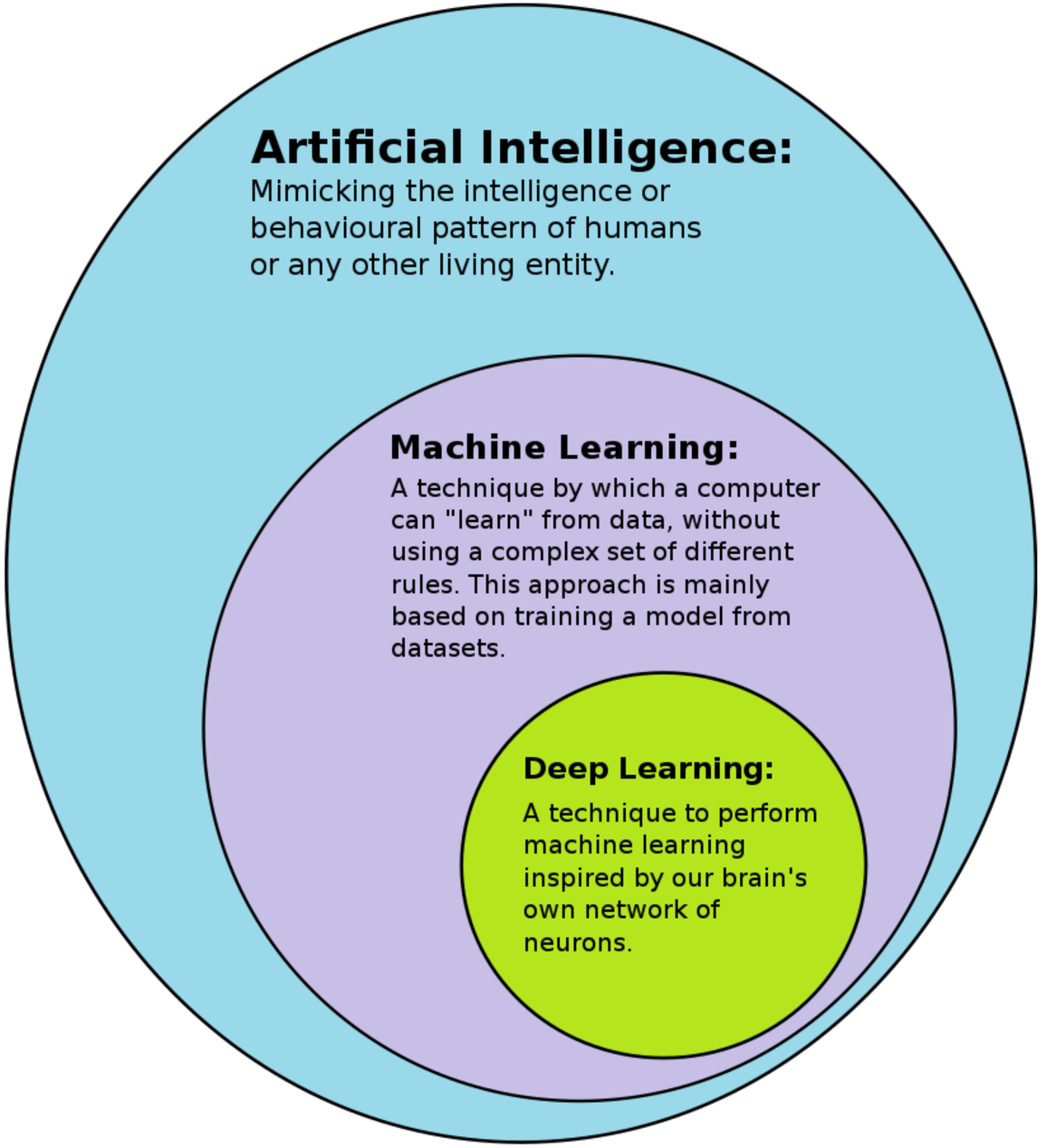
Sub-field of machine learning where algorithms are
- Composed of **parametrized modules.**
- Optimized with **gradient-based methods**.

While traditional ML relies on **hand-crafted features**,

DL autonomously learns hierarchies of features from **unstructured data**: text, audio, images, …

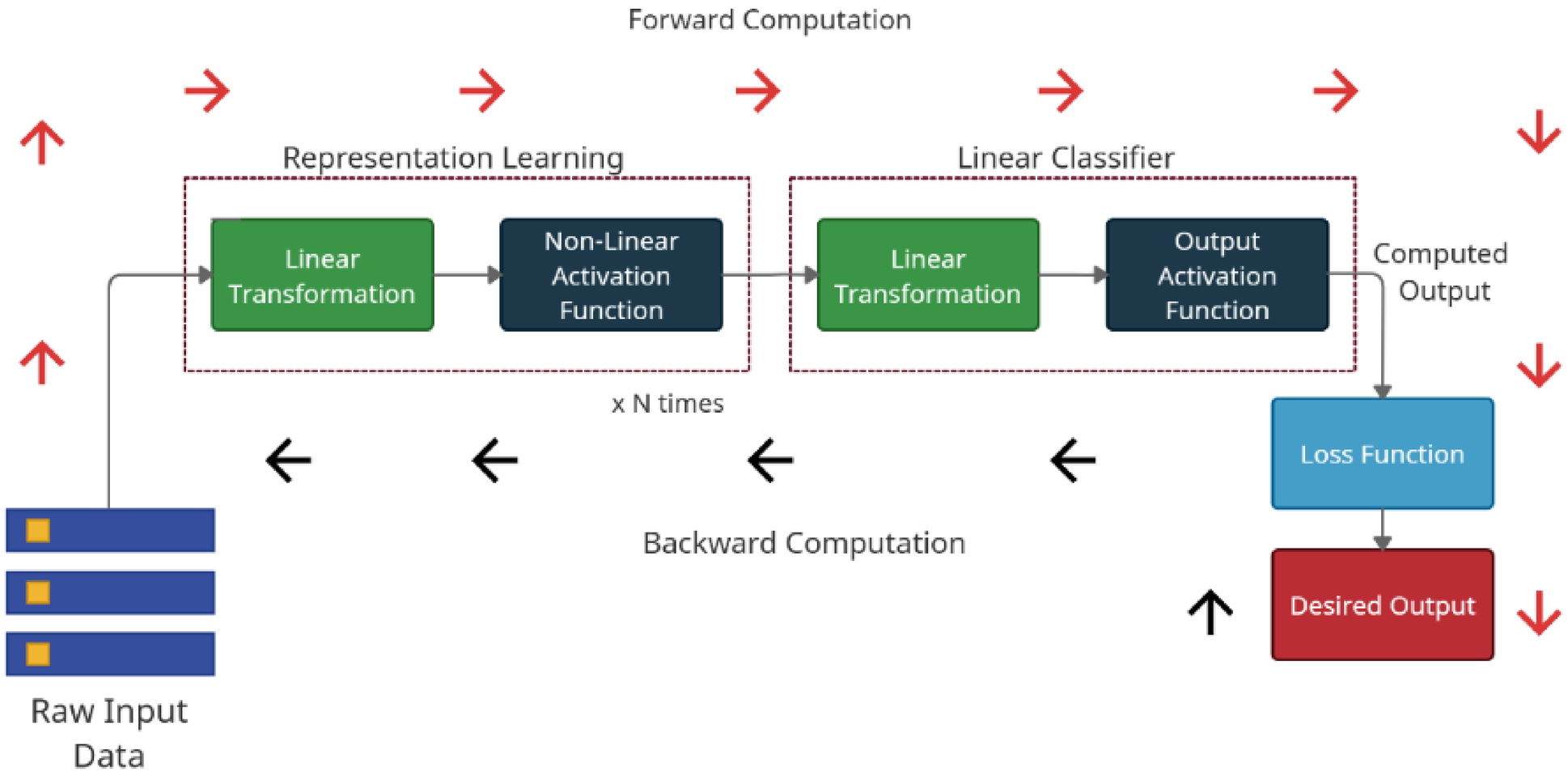


Andrew Ng



Avimanyu786, Wikimedia Commons

Deep learning



Joseph, Manu, and Jeffrey Tackes. Modern Time Series Forecasting with Python. Packt Publishing Ltd, 2024.

Time series require specialized deep learning methods

Time series have **temporal ordering**, **autocorrelation**, and **non-stationarity**.

Forecasting tasks differ from standard ML:

- One-step ahead: sequence-to-one.
- Multi-step ahead: sequence-to-sequence.

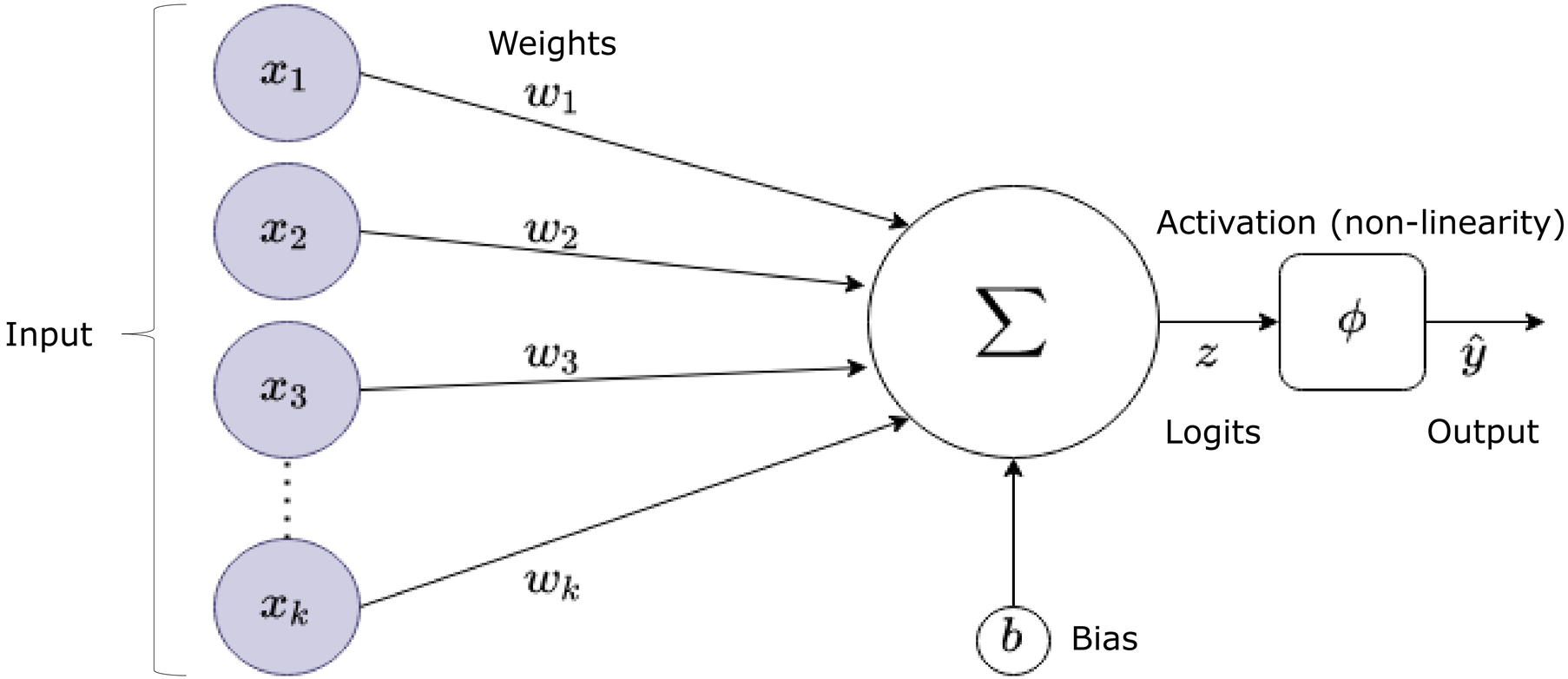
Inputs can include multiple components: **past values**, **covariates**, **static features**, **future-known features**.

- Window-based models: $\hat{y}_{t+1:t+H} = f\left(x_{t-p+1:t}, c_{t-p+1:t}, s\right)$
- Full-sequence models: $\hat{y}_{t+1:t+H} = f(x_{1:t}, c_{1:t}, s)$

DL must model:

- Long-range dependencies
- Seasonality & trends
- Irregular sampling & missing data

Artificial neuron – The Perceptron

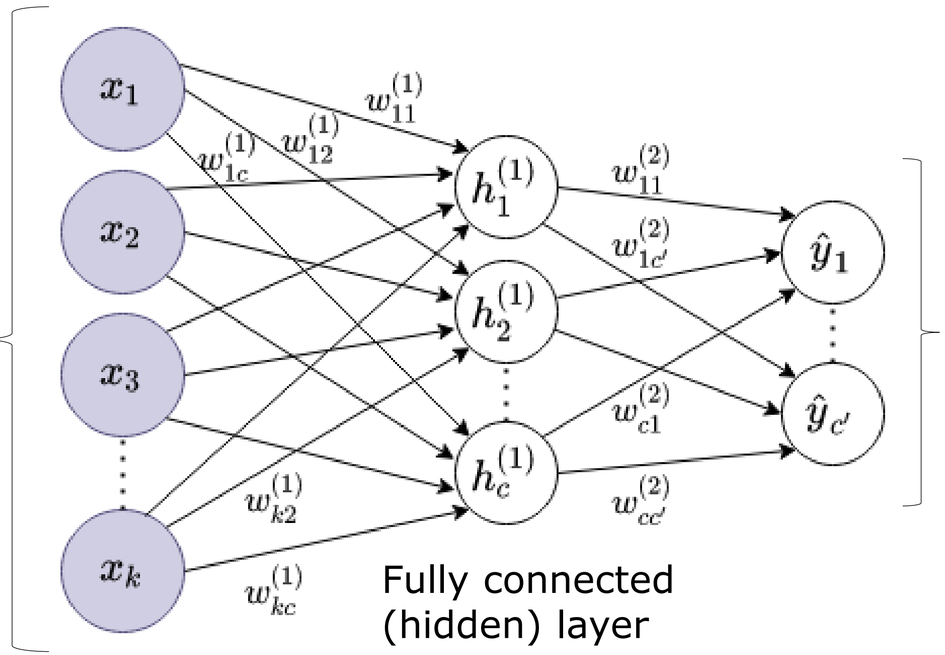


$$\hat{y} = \phi(z) = \phi\left(\sum_{i=1}^{k} w_i x_i + b\right)$$

Training: **adjusting model parameters** so that its output matches the target.

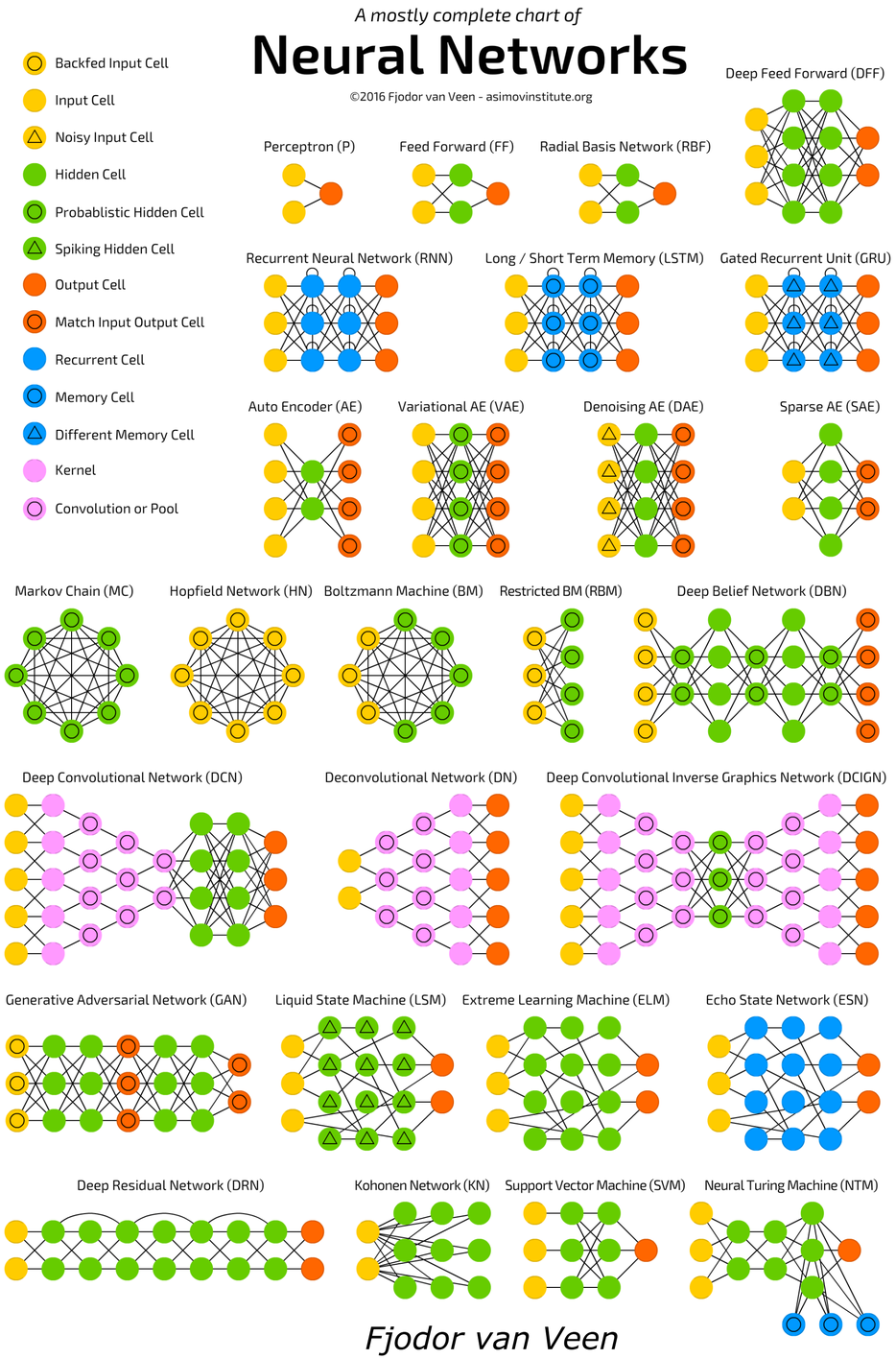
Artificial neural network

Input:
- **Embedded /windowed** time series
- Text
- Images
- Videos
- Audio
- …



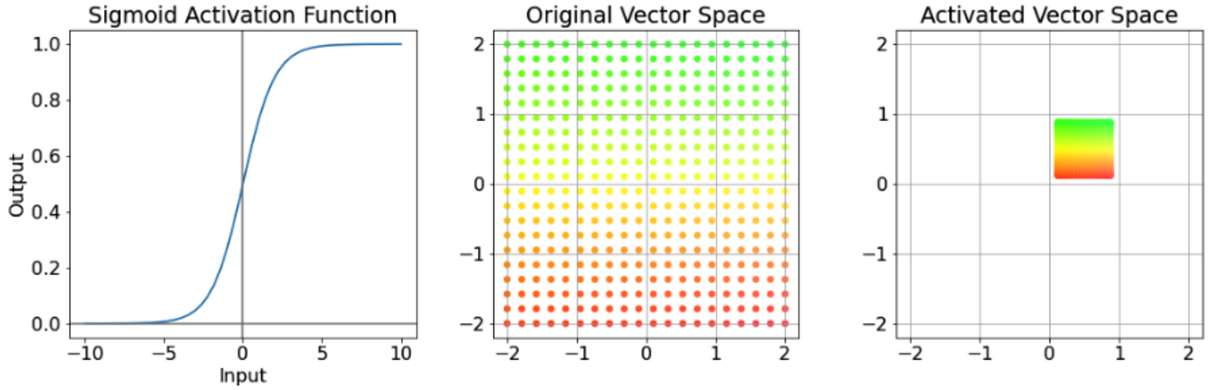
Fully connected (hidden) layer

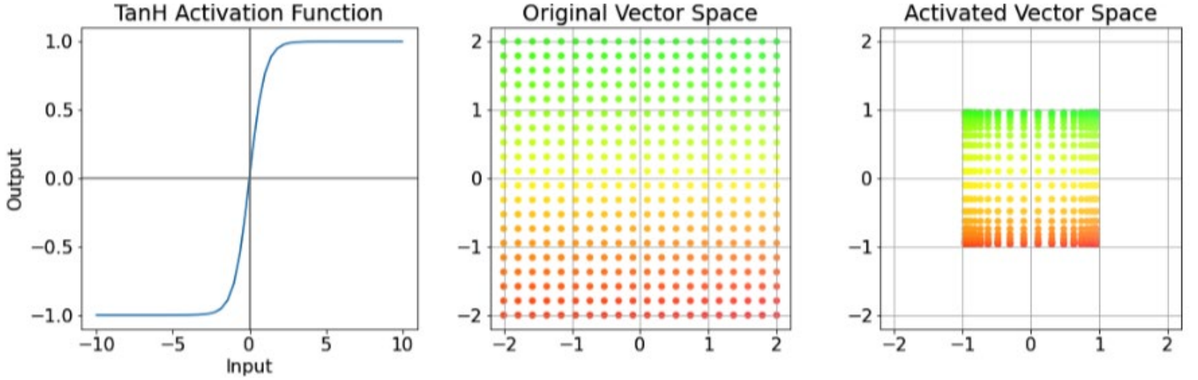
Mild, severe, very severe



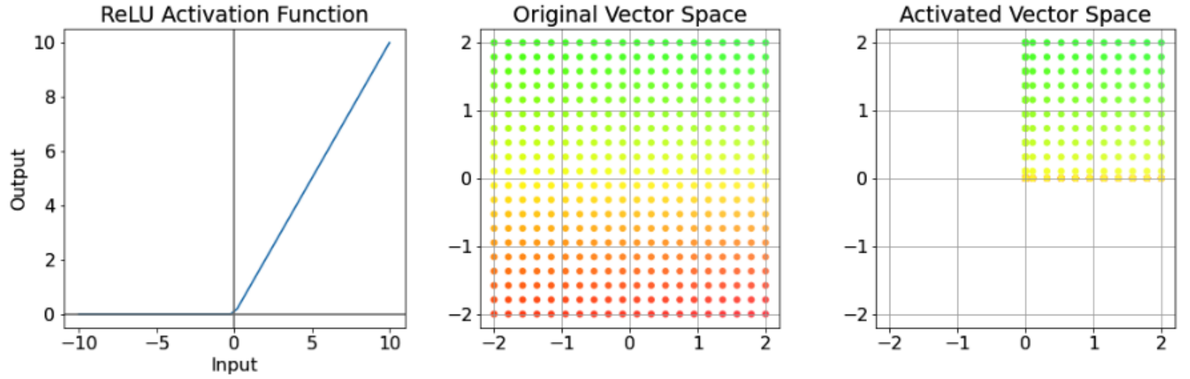
See PyTorch activation functions.

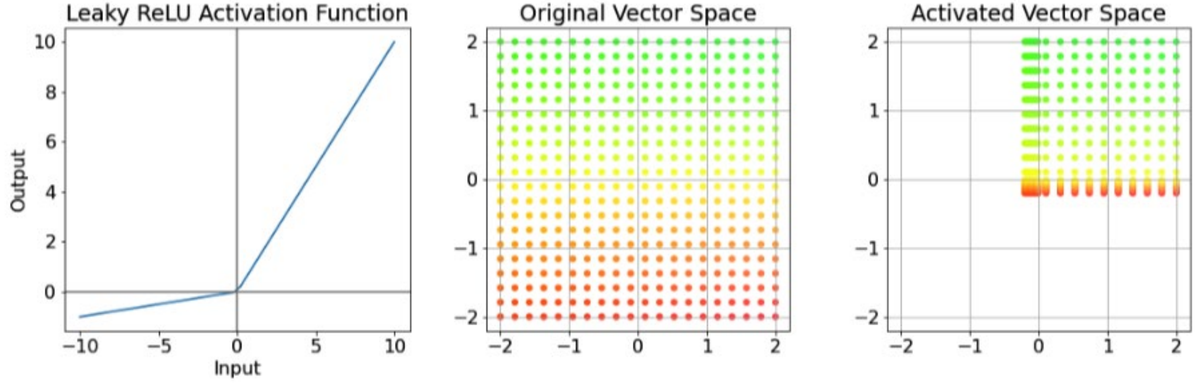
Activation functions





Saturating activation: gradients tend to zero on the flat portions causing the module to **stop learning**.





ReLU unit **stops learning** for negative logits.

Joseph, Manu, and Jeffrey Tackes. Modern Time Series Forecasting with Python. Packt Publishing Ltd, 2024.

Loss functions

Classification

Cross entropy loss:

$$\mathcal{L}_{CE}(\boldsymbol{y}, \hat{\boldsymbol{y}}) = \frac{1}{N}\sum_{i=1}^{N}\sum_{c=1}^{C} y_i^{(c)} \log\left(\hat{y}_i^{(c)}\right)$$

Notation: probability that sample i has label c :

- Ground truth: $y_i^{(c)}$
- Prediction: $\hat{y}_i^{(c)}$

See PyTorch loss functions.

Regression

Mean squared error loss:

$$\mathcal{L}_{MSE}(\boldsymbol{y}, \hat{\boldsymbol{y}}) = \frac{1}{N}\sum_{i=1}^{N}(y_i - \hat{y}_i)^2$$

Model performance is considered **maximized** when the loss is **minimized**:

$$\theta^* = \max_{\theta} \sum_{\forall x \in X} P(t(x), m_\theta(x))$$

$$= \min_{\theta} \sum_{\forall x \in X} \mathcal{L}(\boldsymbol{y}, \hat{\boldsymbol{y}})$$

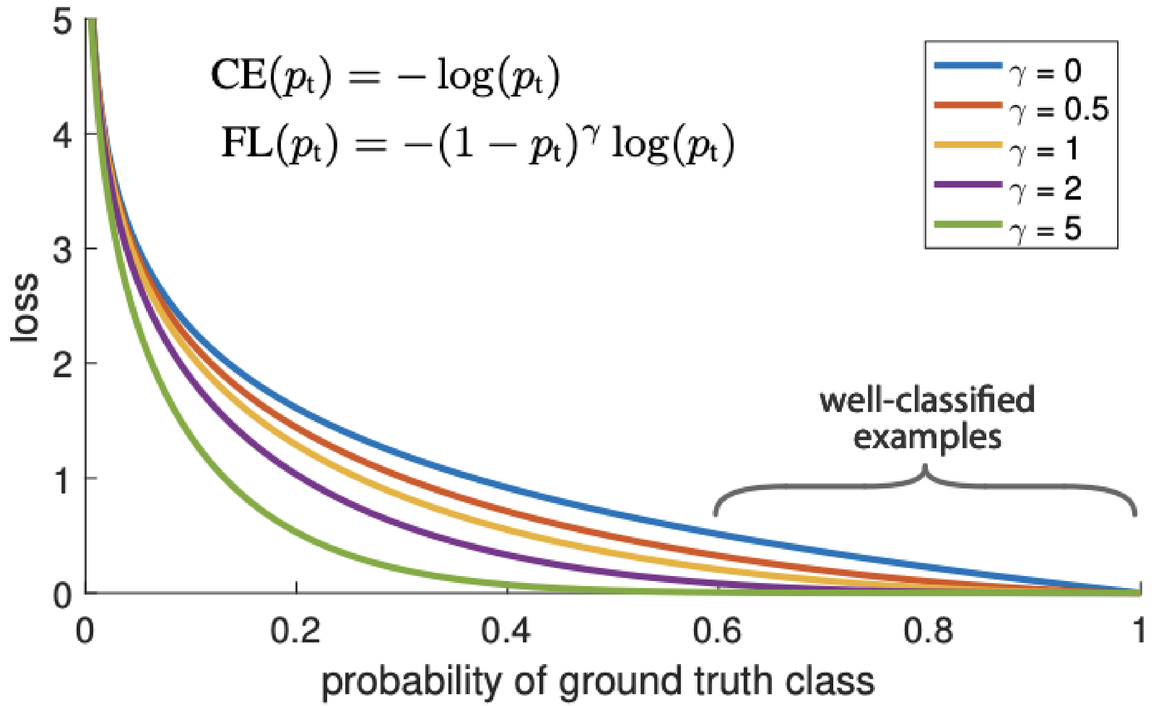
Loss for imbalanced dataset

Focus on **rare classes**:

- Standard loss $L(X, \theta) = \sum_i L(x_i,\ \theta)$
- Weighted loss $L(X, \theta) = \sum_i W_{y_i} L(x_i,\ \theta)$ with $W_C = \frac{|X|}{|\{x_i \in X\ |y_i = C\}|}$

Focus on **hard samples**:

Focal loss, Lin et al. 2018



Limitations of MAE & MSE for time series forecasting

Mean squared error MSE → predicts the **mean**.

Mean absolute error MAE → predicts the **median**.

Both assume symmetric error distributions

- Forecasting errors may be **asymmetric** (e.g., under-forecasting is costlier).

Sensitive to outliers (MSE)

- Squaring error amplifies rare spikes → **unstable** for noisy or heavy-tailed series.

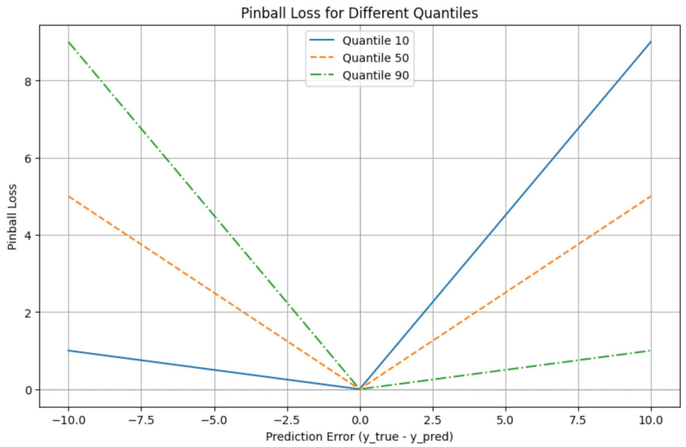
No uncertainty modeling: only returns a single number per horizon.

Multi-horizon issues

- Same loss applied to all future steps → ignores **horizon-dependent difficulty/cost**.

Probabilistic forecasting

Eryk Lewinson



Model learns a representation of the entire **distribution** of possible predicted values.

Quantile (Pinball) loss

- Asymmetric loss function: penalize over- / under-predictions based on quantile q.
- Produces prediction intervals by training multiple quantiles.

$$\mathcal{L}_q(y, \hat{y}_q) = \begin{cases} q(y - \hat{y}_q), & y \geq \hat{y}_q \\ (1-q)(\hat{y}_q - y), & y < \hat{y}_q \end{cases}$$

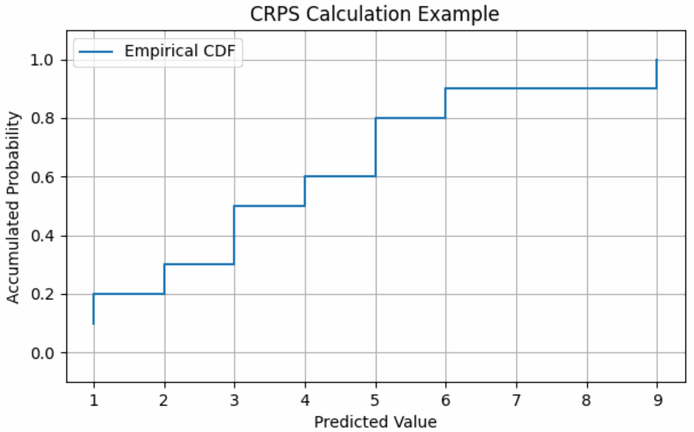
Likelihood-based loss: minimize the negative log-likelihood.

$$\mathcal{L}_{Gaussian}(y, \hat{\mu}, \hat{\sigma}) = \frac{1}{2}\log(2\pi\hat{\sigma}^2) + \frac{(y-\hat{\mu})^2}{2\sigma^2}$$

Continuous ranked probability score (CRPS)

- Average pinball loss over all quantile levels.
- Area between the predicted CDF $F(z)$ and the ideal step CDF.

$$CRPS(F, y) = \int_{-\infty}^{\infty} (F(z) - 1\{z \geq y\})^2 dz$$



Skforecast CRPS

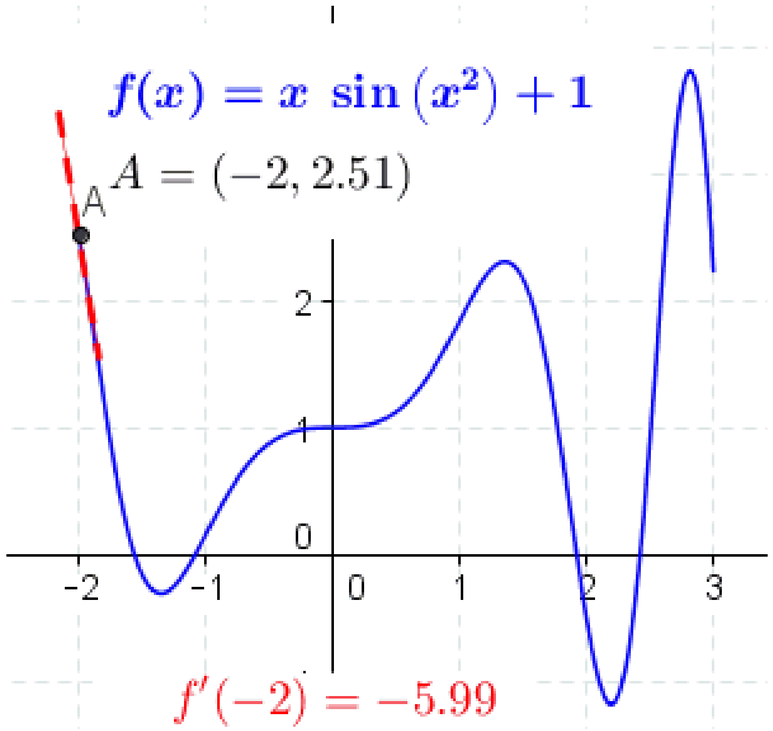
Recap – Derivatives

For a function $f: \mathbb{R} \to \mathbb{R}$, the derivative at $x = x^{(0)}$ is:

$$\frac{\partial f}{\partial x}\left(x^{(0)}\right) = \partial_x f\left(x^{(0)}\right) = \lim_{\epsilon \to 0} \frac{f\left(x^{(0)} + \epsilon\right) - f\left(x^{(0)}\right)}{\epsilon}$$

Interpretation:

- Instantaneous **rate of change** of f at $x = x^{(0)}$.
- Slope of the tangent line at $x = x^{(0)}$.



Dino, Wikimedia

Local **linear approximation**:

$$f\left(x^{(0)} + \Delta x\right) \approx f\left(x^{(0)}\right) + \partial_x f\left(x^{(0)}\right)\Delta x$$

Approximate model locally as a linear function → derivative quantifies the linear **influence of the feature** at $x^{(0)}$.

Recap – Derivatives

Consider the function $f: \mathbb{R} \to \mathbb{R}$

$$f(x) = 3x^2 - 2x + 1$$

The derivative is

$$\partial_x f = 6x - 2$$

At $x^{(0)} = 2$

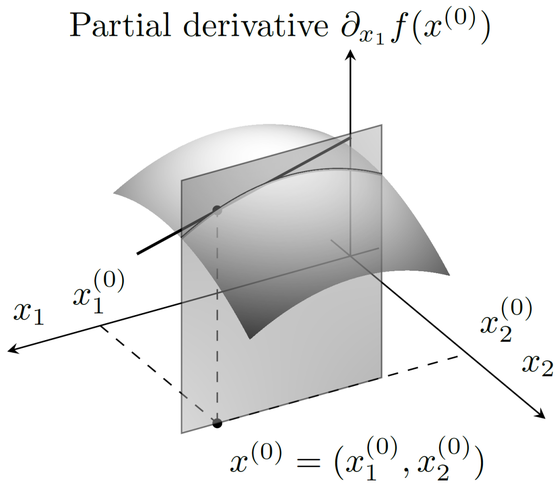
$$\partial_x f(2) = 6 \cdot 2 - 2 = 10$$

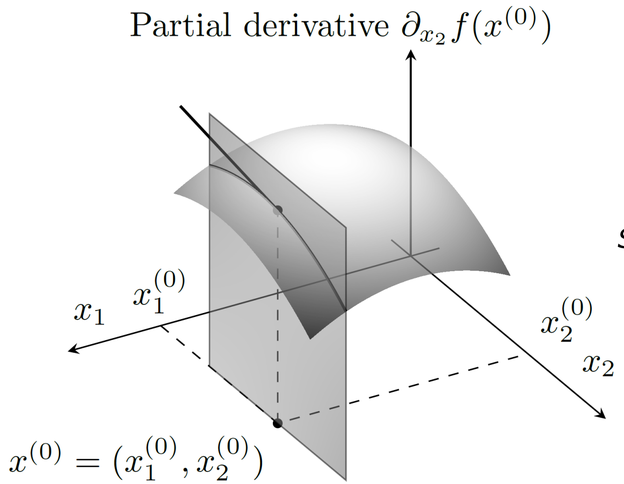
Interpretation: near $x^{(0)} = 2$, increasing x by $\Delta x = 0.01$ increases f by approx 0.1.

Recap – Partial derivatives

For a function $f: \mathbb{R}^2 \to \mathbb{R}$, the partial derivatives at $x = x^{(0)} = \left(x_1^{(0)}, x_2^{(0)}\right)$ are:

$$\partial_{x_1} f\left(x^{(0)}\right) = \lim_{\epsilon \to 0} \frac{f\left(x_1^{(0)} + \epsilon, x_2^{(0)}\right) - f\left(x_1^{(0)}, x_2^{(0)}\right)}{\epsilon}, \qquad \partial_{x_2} f\left(x^{(0)}\right) = \lim_{\epsilon \to 0} \frac{f\left(x_1^{(0)}, x_2^{(0)} + \epsilon\right) - f\left(x_1^{(0)}, x_2^{(0)}\right)}{\epsilon}$$





user121799, StackExchange

Partial derivatives quantify the **isolated** linear **influence per feature** at $x^{(0)}$.

Gradients

For a function $f: \mathbb{R}^{\mathrm{d}} \to \mathbb{R}$, the gradient is the vector of partial derivatives:

$$\nabla_x f\left(x^{(0)}\right) = \left(\partial_{x_1} f\left(x^{(0)}\right), \ldots, \partial_{x_d} f\left(x^{(0)}\right)\right)^\top$$

Interpretation:

- Direction of the steepest increase of f at $x^{(0)}$.
- The magnitude quantifies the overall strength of the linear influence at $x^{(0)}$.

Local **linear approximation** of the model:

$$f\left(x^{(0)} + \Delta x\right) \approx f\left(x^{(0)}\right) + \nabla_x f\left(x^{(0)}\right)^\top \Delta x$$

When training a model, we compute the gradient with respect to its **parameters** $\nabla_\theta f\left(x^{(0)}\right)$.

For feature attribution, we compute the gradient with respect to the **input** $\nabla_x f\left(x^{(0)}\right)$.

Partial derivatives and gradients

Consider the function $f: \mathbb{R}^2 \to \mathbb{R}$

$$f(x_1, x_2) = 2x_1^2 + 3x_2$$

The partial derivatives are

$$\partial_{x_1} f = 4x_1, \qquad \partial_{x_2} f = 3$$

The gradients vector is

$$\nabla_x f = (4x_1, 3)^\top$$

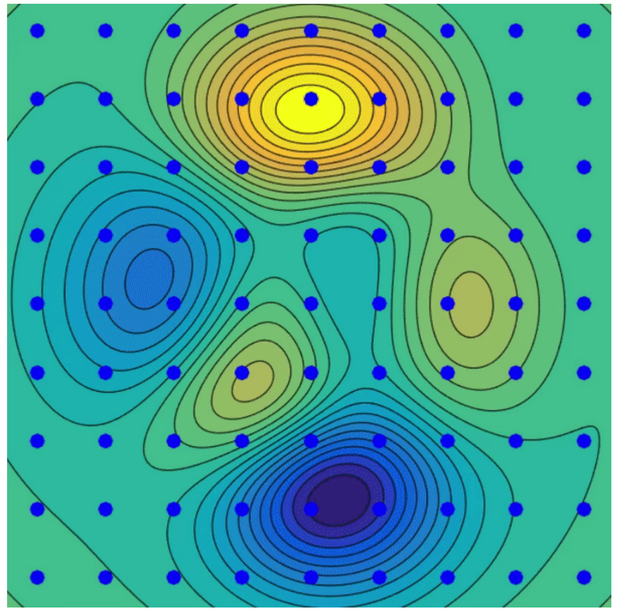
At $x^{(0)} = (1,2)$

$$\nabla_x f(1,2) = (4,3)^\top$$

Interpretation: near $x^{(0)} = (1,2)$, increasing x_1 changes output more than changing x_2 with ratio $4:3$.

Gradient descent & variants

Gpeyre, wikimedia



Find **local minimum** of a differentiable function

- Iteratively update parameters in the opposite direction of the gradients.

The optimizer defines the **update rule**, given

- Learning rate: η
- Network parameters at step t: θ_t
- Loss gradient: $\nabla\mathcal{L}$
- Hyper parameters: $\beta_1,\ \beta_2,\ \ldots$

Gradient descent

$$\theta_{t+1} = \theta_t - \eta\nabla\mathcal{L}(\theta_t)$$

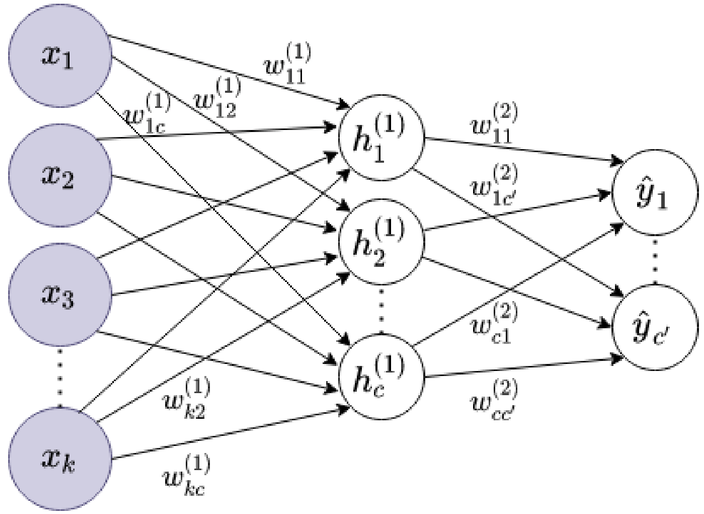
Momentum

$$\mathbf{m}_{t+1} = \beta_1\mathbf{m}_t + (1-\beta_1)\nabla\mathcal{L}(\theta_t)$$
$$\theta_{t+1} = \theta_t - \eta\,\mathbf{m}_{t+1}$$

Adam *(adaptive moment estimation optimizer)*

$$\mathbf{m}_{t+1} = \beta_1\mathbf{m}_t + (1-\beta_1)\nabla\mathcal{L}(\theta_t)$$
$$\mathbf{v}_{t+1} = \beta_2\mathbf{v}_t + (1-\beta_2)(\nabla\mathcal{L}(\theta_t))^2$$
$$\theta_{t+1} = \theta_t - \eta\frac{\mathbf{m}_{t+1}/(1-\beta_1^{t+1})}{\sqrt{\mathbf{v}_{t+1}/(1-\beta_2^{t+1})}}$$

Training – Forward & backward propagation



1. **Forward propagation**: compute network predictions.

$$\mathbf{h}^{(1)} = \phi^{(1)}(\mathbf{z}^{(1)}) = \phi^{(1)}({\mathbf{W}^{(1)}}^{\top}\mathbf{x} + \mathbf{b}^{(1)})$$

$$\hat{\mathbf{y}} = \phi^{(2)}({\mathbf{W}^{(2)}}^{\top}\mathbf{h}^{(1)} + \mathbf{b}^{(2)})$$

2. Compute the network **loss** $\mathcal{L}$.

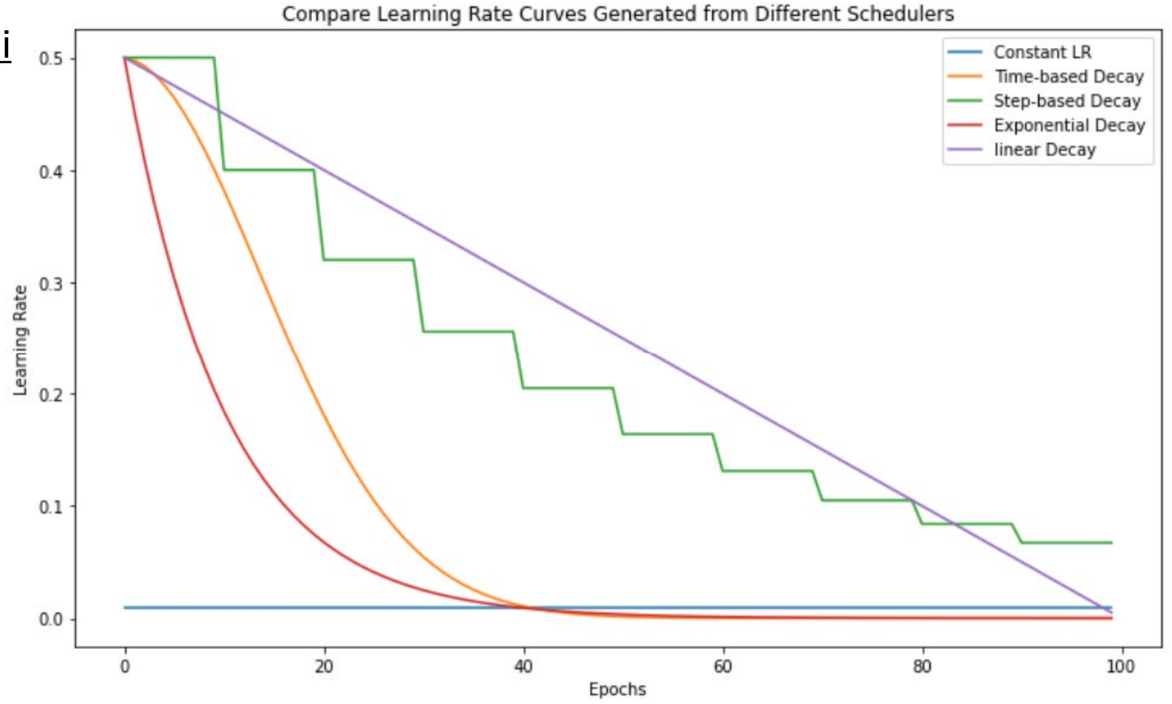
3. **Backward propagation**: compute the loss gradients with respect to network's parameters.
   - The forward propagation is a **composition of differentiable operations** → the **chain rule** can be applied.

$$\frac{\partial \mathcal{L}}{\partial w_{11}^{(1)}} = \left(\sum_{j=1}^{c'} \frac{\partial \mathcal{L}}{\partial \phi^{(2)}} \times \frac{\partial \phi^{(2)}}{\partial z_j^{(2)}} \times \frac{\partial z_j^{(2)}}{\partial \phi^{(1)}}\right) \times \frac{\partial \phi^{(1)}}{\partial z_1^{(1)}} \times \frac{\partial z_1^{(1)}}{\partial w_{11}^{(1)}}$$

$$\frac{\partial \mathcal{L}}{\partial b_1^{(1)}} = \left(\sum_{j=1}^{c'} \frac{\partial \mathcal{L}}{\partial \phi^{(2)}} \times \frac{\partial \phi^{(2)}}{\partial z_j^{(2)}} \times \frac{\partial z_j^{(2)}}{\partial \phi^{(1)}}\right) \times \frac{\partial \phi^{(1)}}{\partial z_1^{(1)}} \times \frac{\partial z_1^{(1)}}{\partial b_1^{(1)}}$$

Learning rate scheduling

neptune.ai



Choosing the learning rate is a trade-off

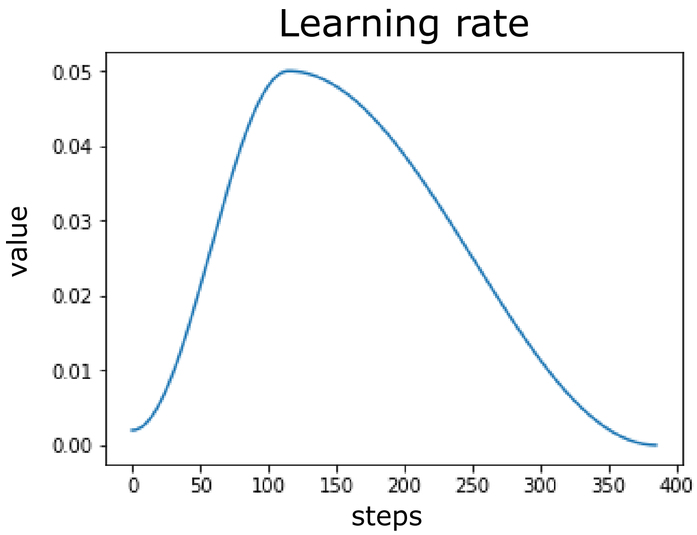
- Convergence speed
- Reaching the global minima

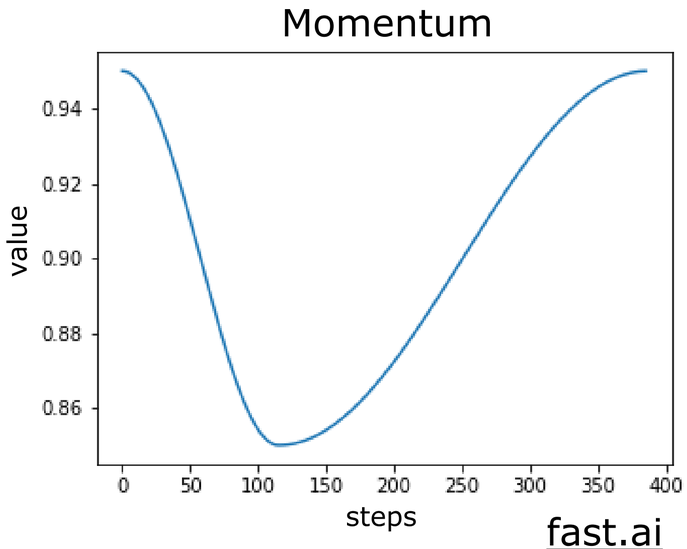
Learning rate **decay**.

Learning rate **warmup**.

Cyclical learning **schedules**

- 1-cycle policy
- Cosine annealing scheduler





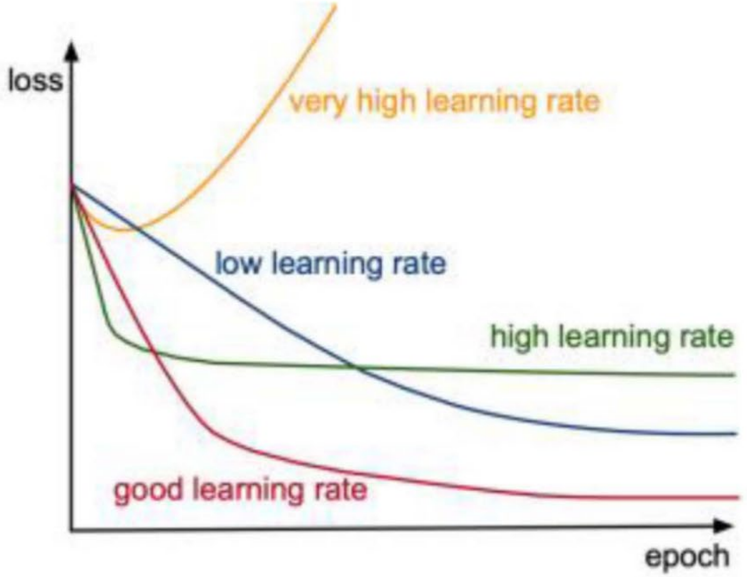
fast.ai

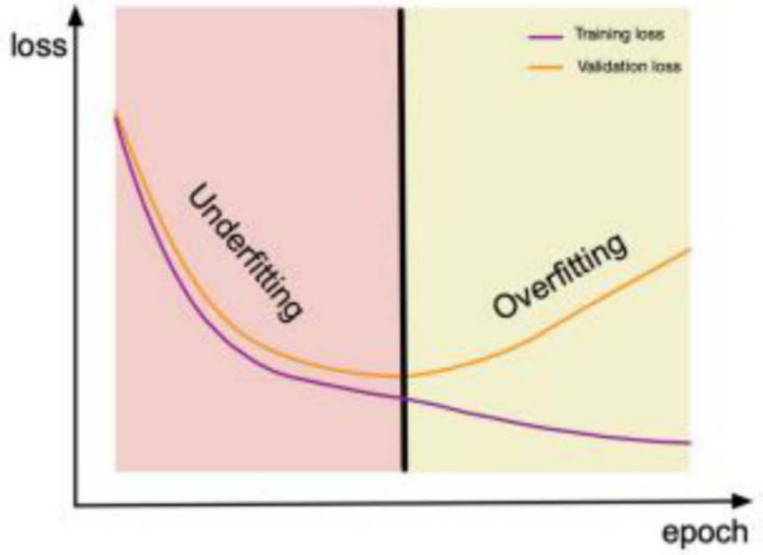
Training monitoring

Monitor train and validation losses

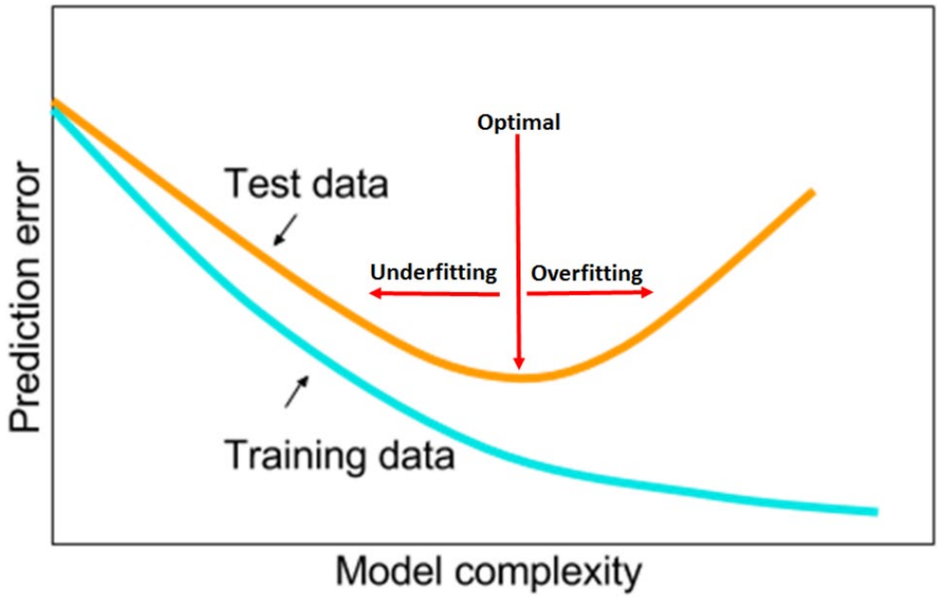
Use experiment tracker: Mlflow, DVC, Weights & Biases, Neptune, Tensorboard.

Torres-Velázquez et al., 2020



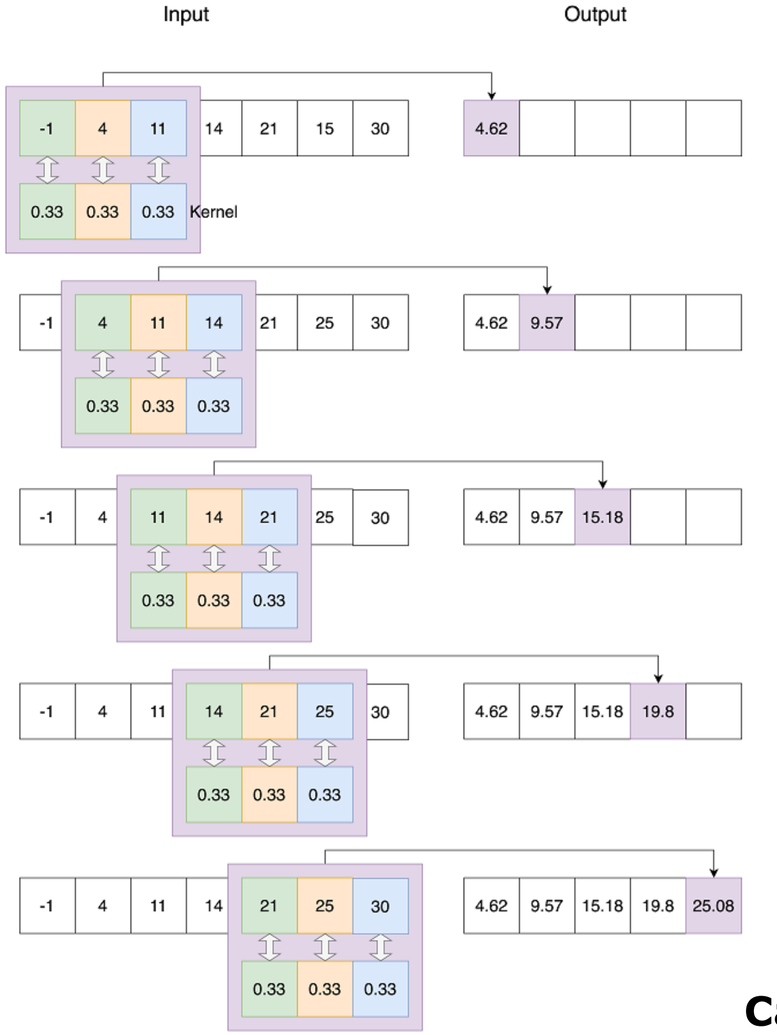


Leslie, 2018

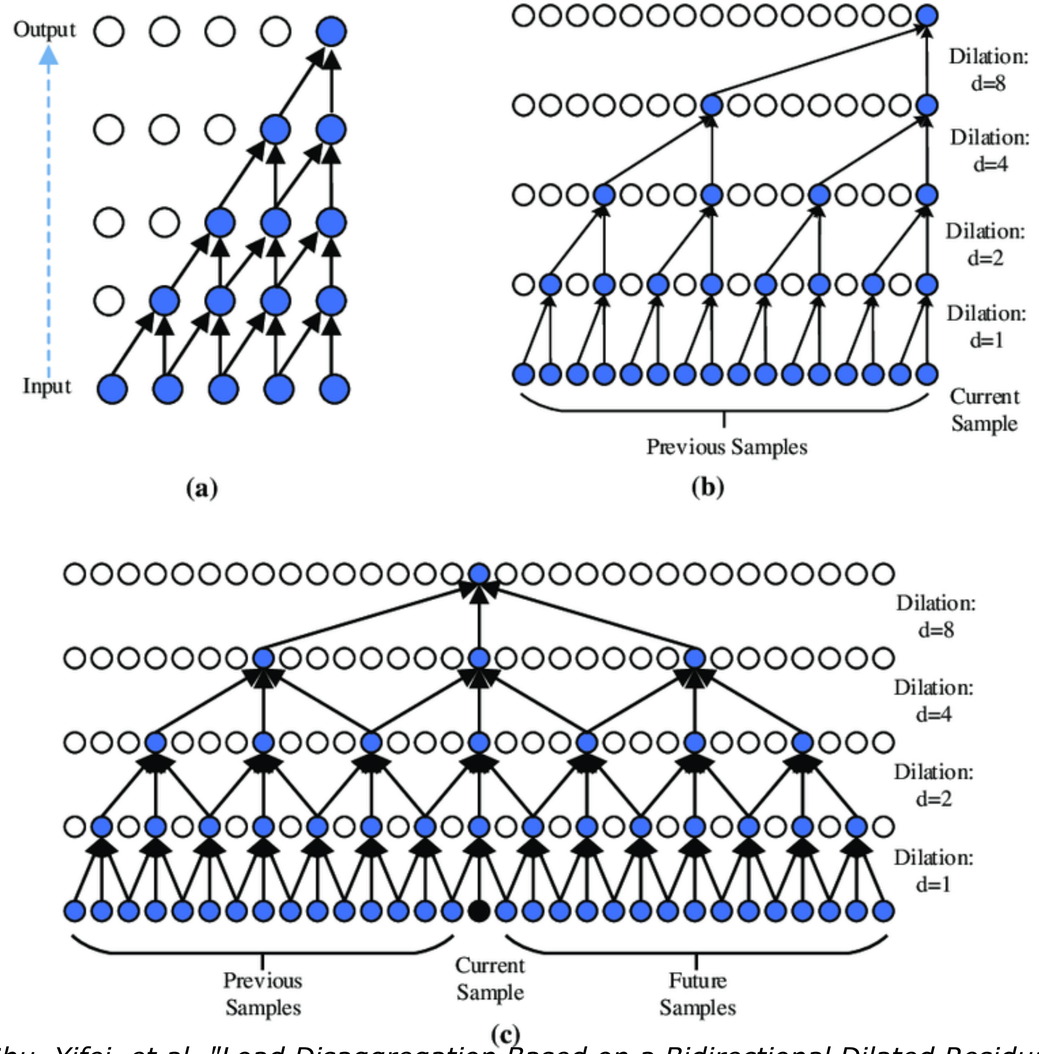


1D Convolution

Stride = 1
Kernel size = 3



Chaudhury, Krishnendu, et al. Math and Architectures of Deep Learning. Simon and Schuster, 2024.

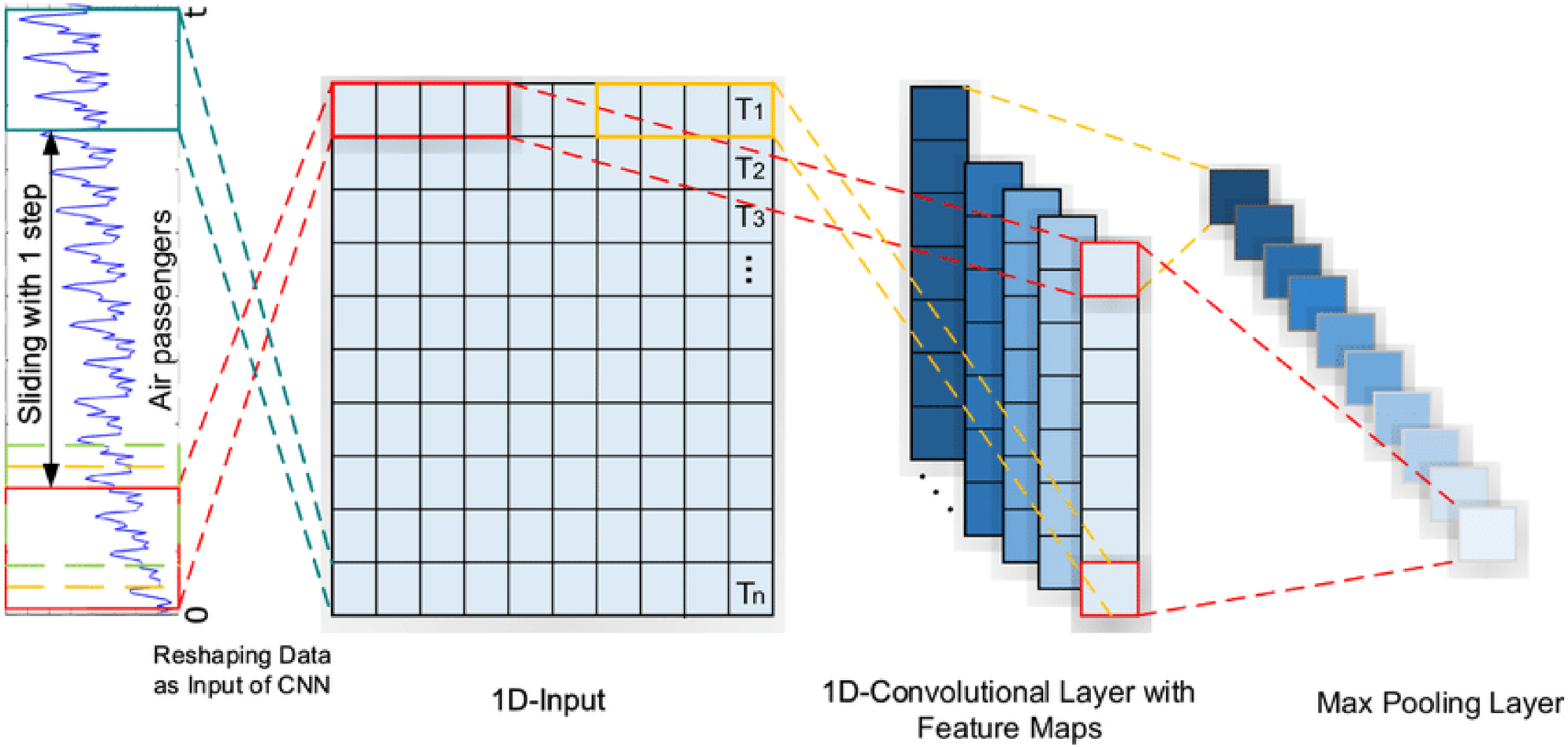


Shu, Yifei, et al. "Load Disaggregation Based on a Bidirectional Dilated Residual Network with Multihead Attention." Electronics 12.12 (2023): 2736.

Causal convolution: ensure that no future time steps is used to predict the current timestep.

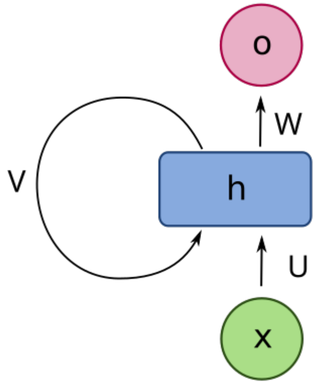
Receptive field: the region in the input associated with a feature.

Temporal convolutional networks (TCN)



Wu, Xinfang, et al. "Forecasting air passenger traffic flow based on the two-phase learning model." The Journal of Supercomputing 77 (2021): 4221-4243.

Recurrent neural network (RNN)

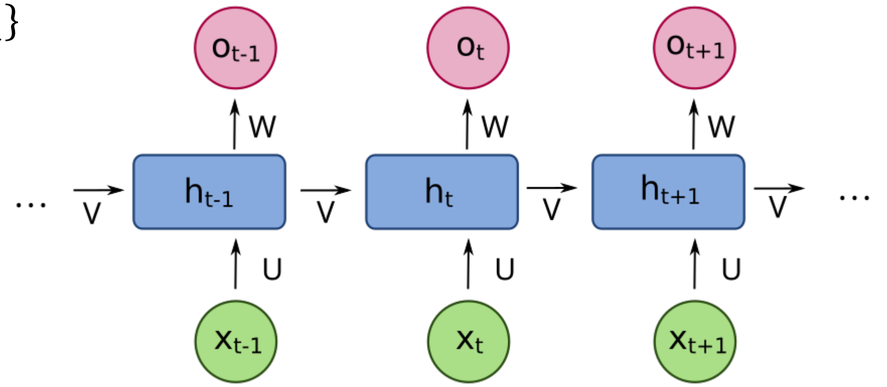


Fdeloche, wikimedia

RNN are designed to handle **sequential data**.

RNN maps the **input sequence** $\{x_1, \dots x_n\}$ to an **output sequence** $\{o_1, \dots o_n\}$ based on a **hidden state sequence** $\{h_0, h_1, \dots h_n\}$.

- h_0 is typically initialized with zeros.



At each time step,

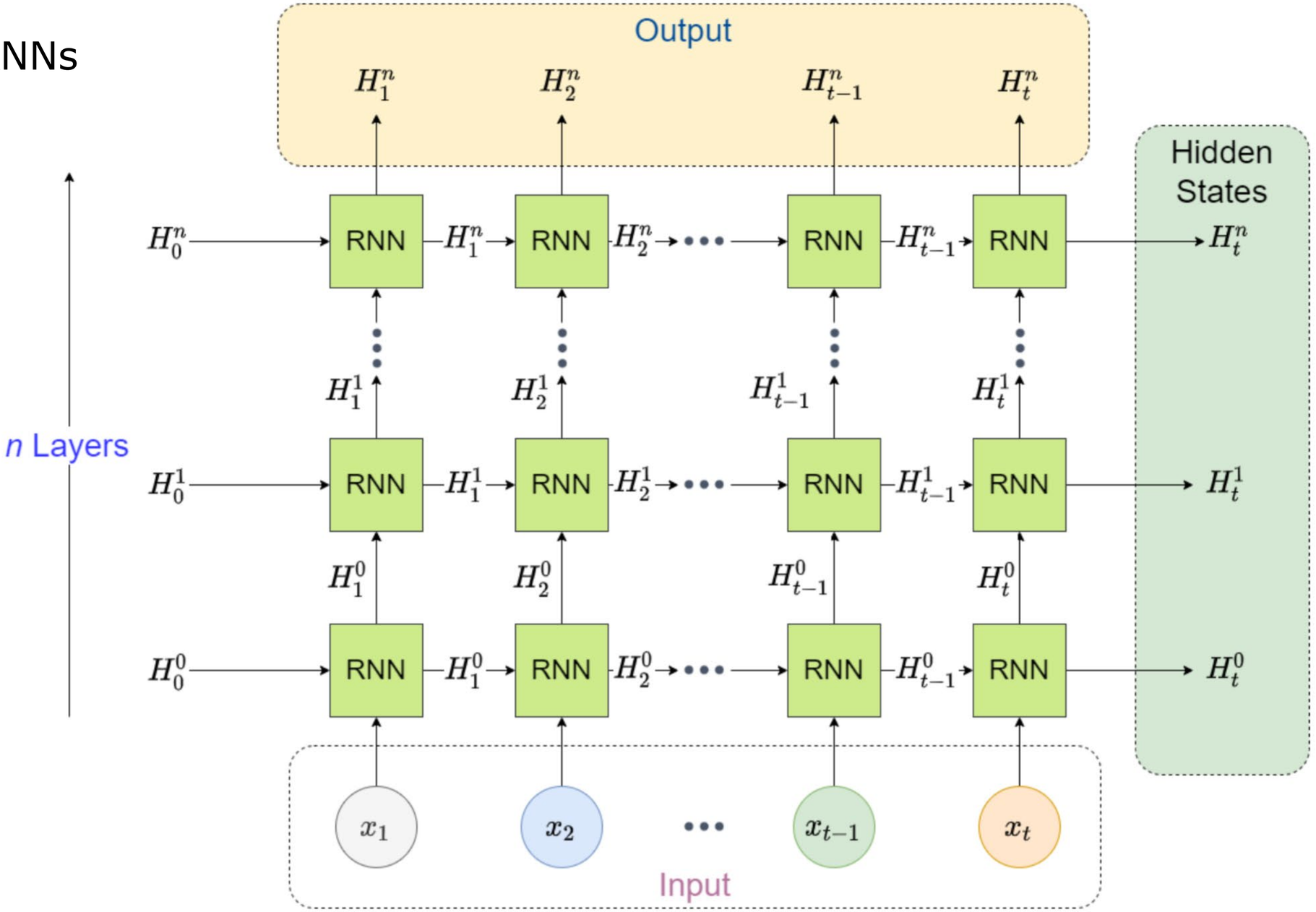
$$h_t = \phi_h(Ux_t + Vh_{t-1} + b_h)$$
$$o_t = \phi_o(Wh_t + b_o)$$

with U, V, W **learnable** weight matrices and b_h, b_o **learnable** bias vectors.

Backpropagate gradients within single unit through time steps.

- **Vanishing/exploding gradient** problem with long sequences.

Stacking RNNs



Joseph, Manu, and Jeffrey Tackes. Modern Time Series Forecasting with Python. Packt Publishing Ltd, 2024.

Long Short-Term Memory (LSTM)

Forget gate: what information can be discarded.

$$F_t = \sigma\big(W_f[h_{t-1}, x_t] + b_f\big)$$

Input gate: what information should be added.

$$I_t = \sigma(W_i[h_{t-1}, x_t] + b_i)$$

Candidate memory cell state:

$$\tilde{c}_t = \tanh(W_{\tilde{c}}[h_{t-1}, x_t] + b_{\tilde{c}})$$

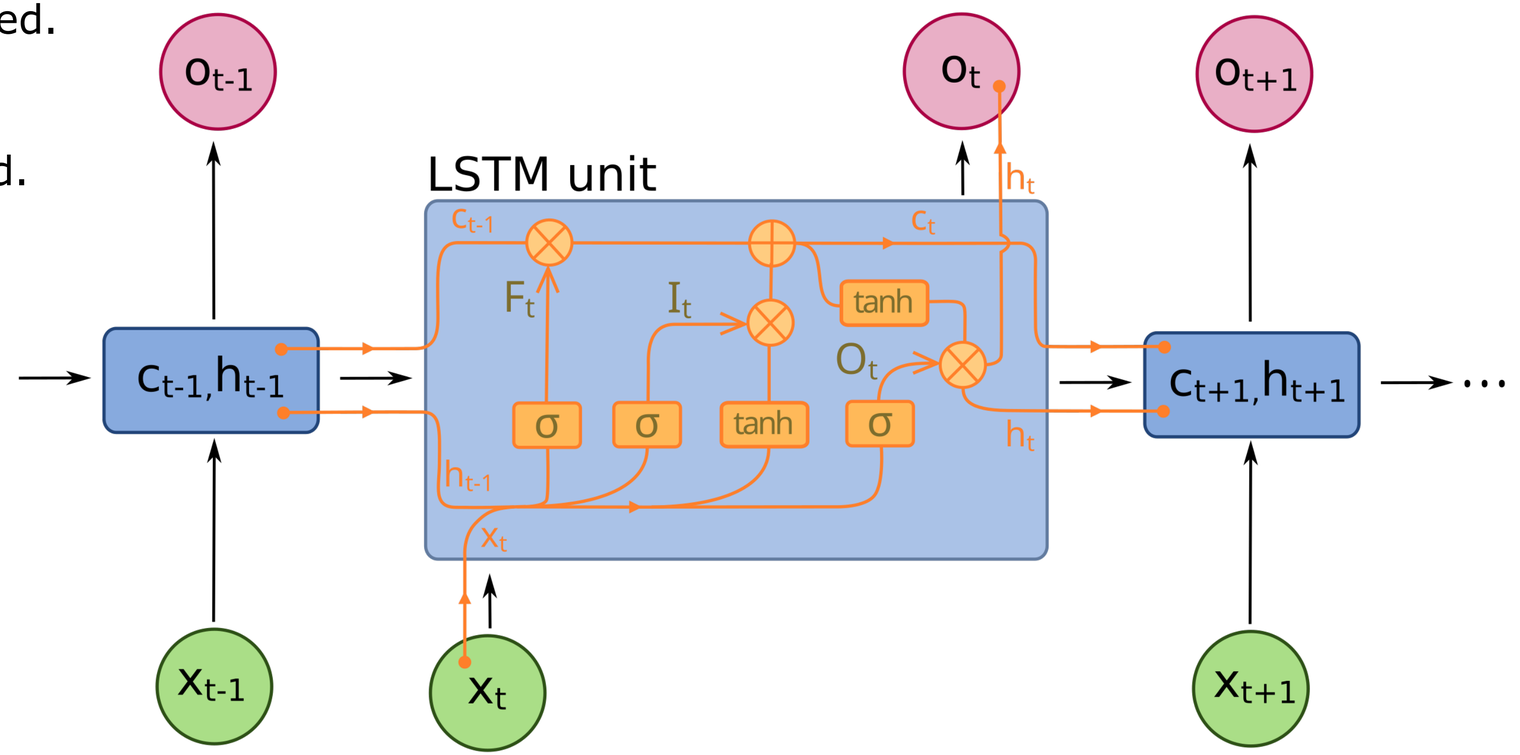
Output gate:

$$O_t = \sigma(W_o[h_{t-1}, x_t] + b_o)$$

Cell state:

$$c_t = F_t \otimes c_{t-1} + I_t \otimes \tilde{c}_t$$
$$h_t = O_t \otimes \tanh(c_t)$$

Fdeloche, wikimedia



The cell output is its state h_t. The memory cell c_t serves as long-term memory and "gradient highway".

$W_{\dots}$ are **learnable** weight matrices and $b_{\dots}$ **learnable** bias vectors.

$\otimes$ is elementwise multiplication.

σ is the sigmoid activation.

$[h_{t-1}, x_t]$ represents concatenation.

Gated recurrent unit (GRU)

Simplification of LSTM:

- No memory cell.
- Use hidden state h_t as "gradient highway".

Reset gate:

$$R_t = \sigma(W_r[h_{t-1}, x_t] + b_r)$$

Update gate:

$$Z_t = \sigma(W_z[h_{t-1}, x_t] + b_z)$$

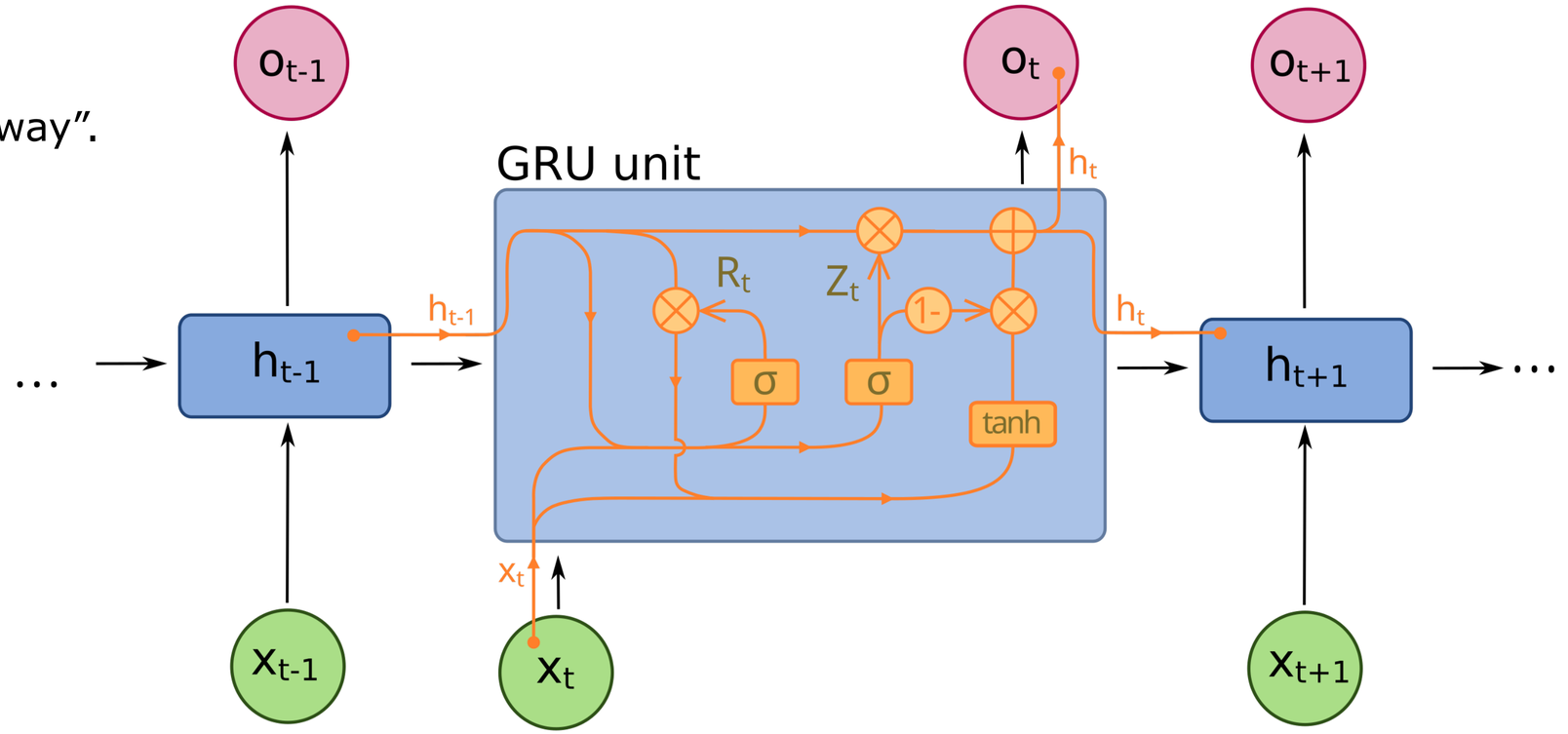
Candidate hidden state:

$$\tilde{h}_t = \tanh\left(W_{\tilde{h}}[R_t \otimes h_{t-1}, x_t] + b_{\tilde{h}}\right)$$

Cell state:

$$h_t = Z_t \otimes h_{t-1} + (1 - Z_t) \otimes \tilde{h}_t$$

Fdeloche, wikimedia



$\otimes$ is elementwise multiplication.

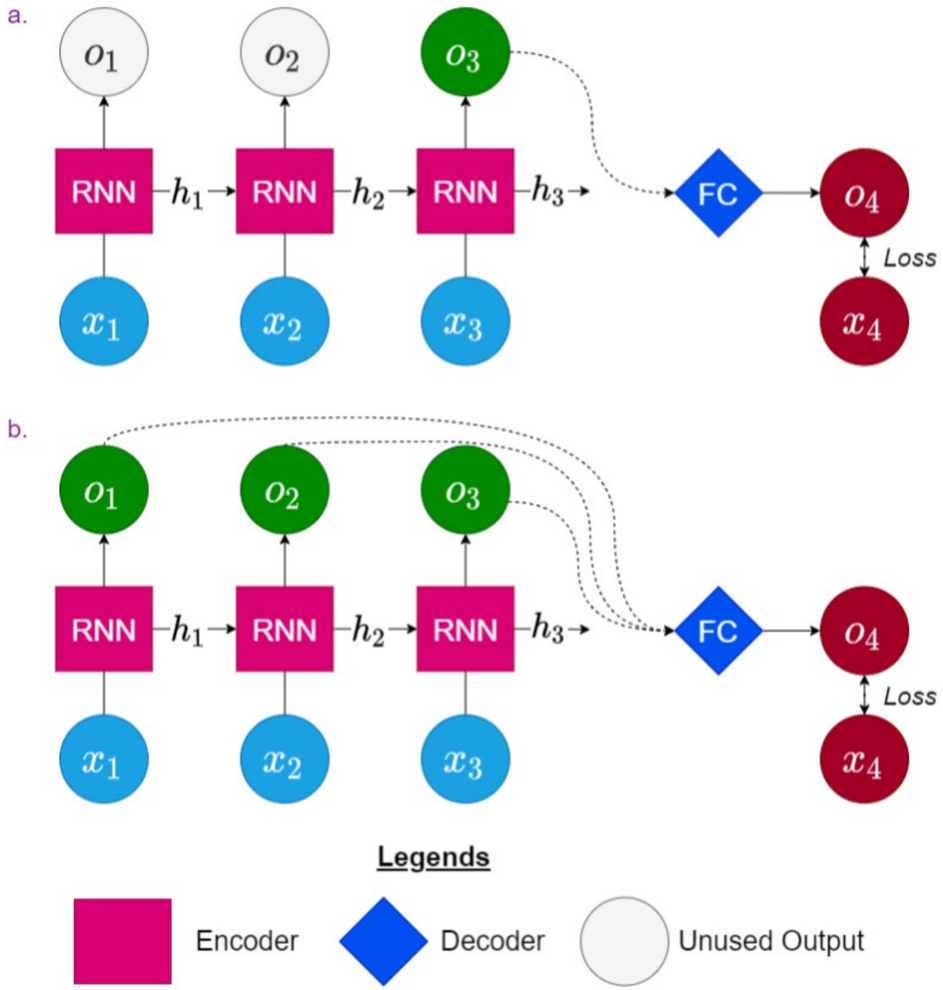
σ is the sigmoid activation.

$[h_{t-1}, x_t]$ represents concatenation.

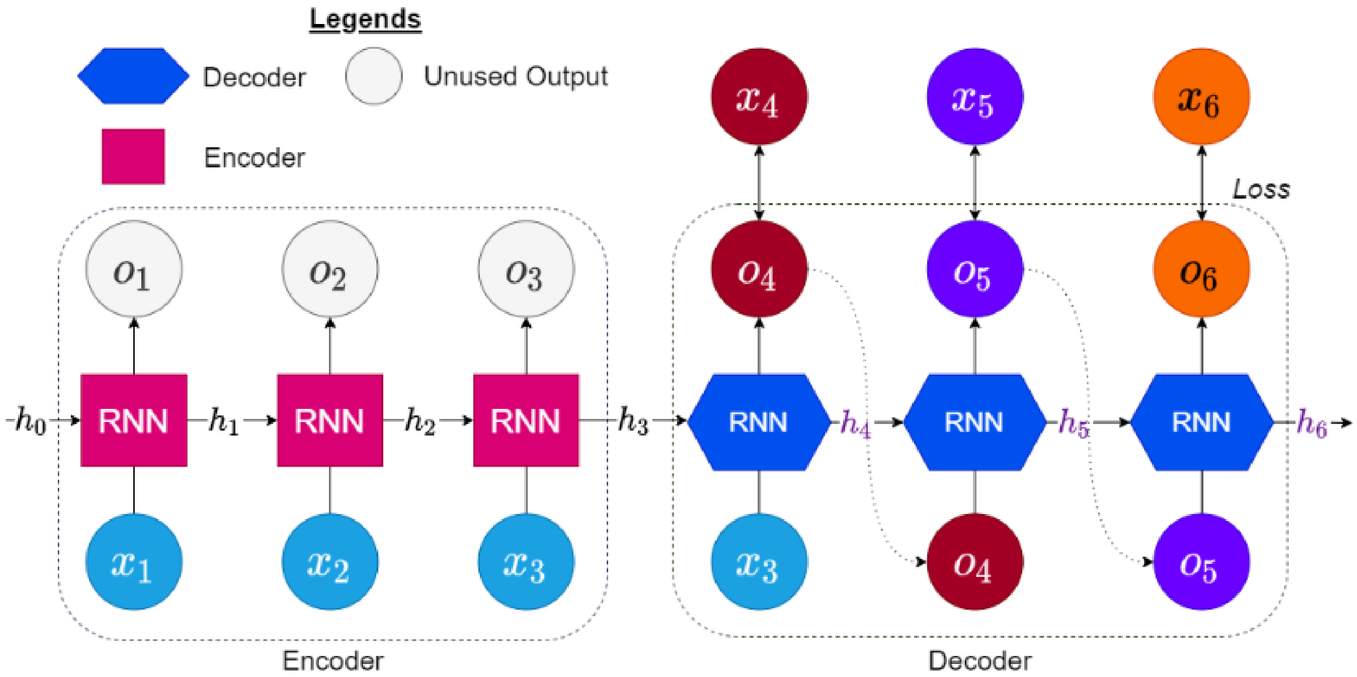
$W_{\dots}$ are **learnable** weight matrices and $b_{\dots}$ **learnable** bias vectors.

Encoder-decoder architecture

Sequence to one

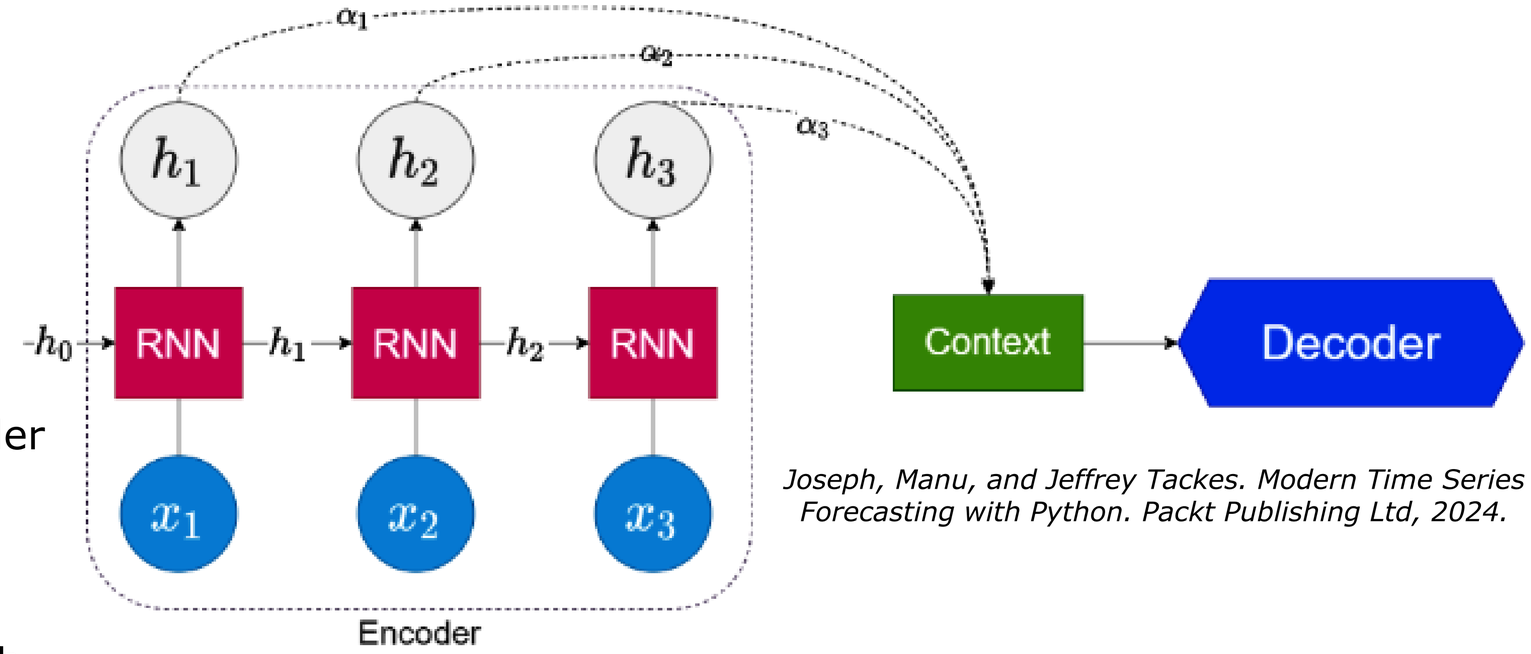


Sequence to sequence



Joseph, Manu, and Jeffrey Tackes. Modern Time Series Forecasting with Python. Packt Publishing Ltd, 2024.

Attention



Joseph, Manu, and Jeffrey Tackes. Modern Time Series Forecasting with Python. Packt Publishing Ltd, 2024.

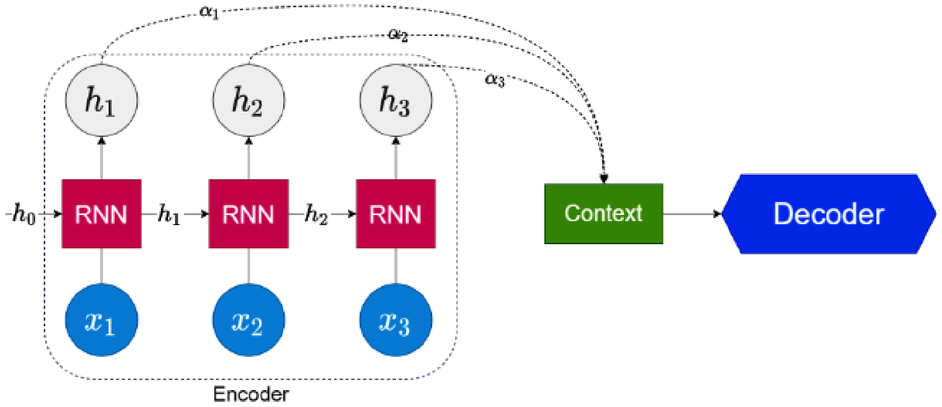
Using only the last hidden state of the encoder causes an **information bottleneck**.

Instead, learn **attention weights** α_i for each hidden state and combine them into a single **context vector** while decoding.

Attention is an internal signal describing **interactions across input positions**.

- How much a part of the input should attend to every other part.
- How does the model distribute its **focus**?
- Reveals **where** the model focuses in the input.
- Gives an **interpretable** distribution.

Generalized attention model



Manu et al. "Modern Time Series Forecasting with Python". Packt Publishing Ltd, 2024.

$$A(q_i, K, V) = \sum_j \underbrace{p\left(a(q_i, k_j)\right)}_{\alpha_{ij}} v_j$$

with

- Queries $\mathrm{Q} = \{q_i\}$: e.g., decoder states or current token representation.
- Keys $K = \{k_j\}$: candidate information sources e.g., encoder state.
- Values $V = \{v_j\}$: semantic content associated with each key.
- Alignment function $a(q_i, k_j)$: similarity score e.g., dot product, MLP.
- Distribution function p: normalizes scores into attention weights e.g., softmax.

Attention forms a **probability distribution** over values.

For query q_i, the weights α_{ij} sum to 1 and reflect the **similarity** with the keys.

High weight ⇒ the model **focuses** strongly on that value.

Output is the weighted average of values, i.e., the **expected value** under the learned attention distribution.

Scaled dot-product attention (Transformer)

$$a(q_i, k_j) = \frac{q_i^\top k_j}{\sqrt{d_k}}, \qquad \alpha_{ij} = \frac{\exp\left(a(q_i, k_j)\right)}{\sum_{j'} \exp\left(a(q_i, k_{j'})\right)}, \qquad A(q_i, K, V) = \sum_j \alpha_{ij} v_i$$

with d_k the key/query projection dimension.

Keys and queries are projected from **token embeddings**.
Their dot products give similarity scores.

Scaling **stabilizes gradients** when key/query vectors are high-dimensional.
Softmax converts scores into a probability distribution.

The output is a **convex combination** (weighted sum with non-negative weights summing to one) of value vectors.

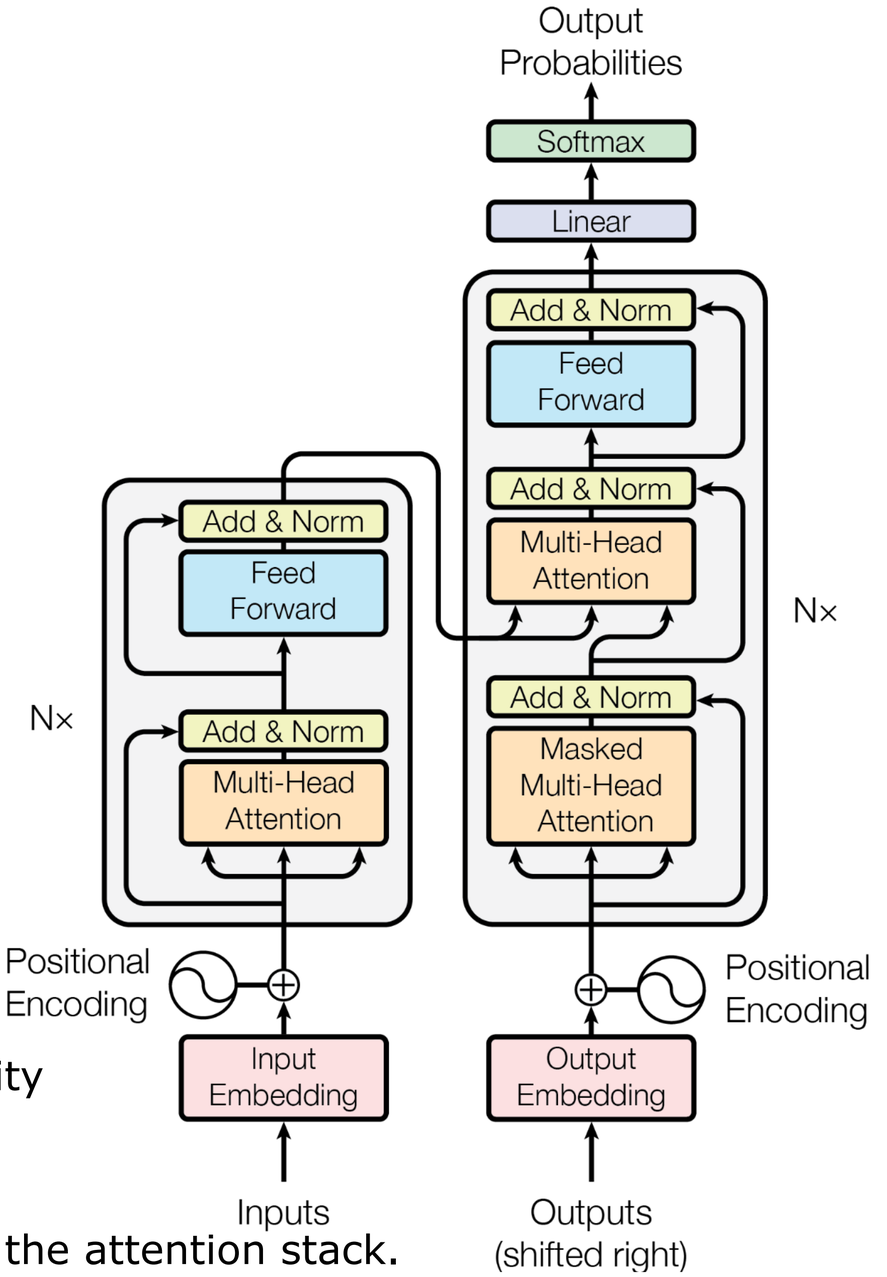
- Remains within values convex hull i.e., never extrapolates outside their span.

Transformers – Attention is all you need

Due to their recurrence, RNNs cannot be parallelized.

Transformers avoid recurrence altogether and process sequences in parallel.

- **Positional encoding** to retain positional (temporal) information.
- **Skip connections** to improve gradient flow through the network.
- **Layer normalization** for faster training and some regularization.
- **Multi-head self-attention (MHA)**: attention scores between input sequence and itself.
- **Masked MHA**: mask information from the future timesteps.



Vanilla transformers **struggle** with long sequences due to quadratic complexity

- Patch embeddings: embed window of consecutive timesteps.
- Sparse attention: compute only subset of attention connections.
- Multi-scale temporal kernels: convolutional/spectral filters before/inside the attention stack.

Vaswani, A. "Attention is all you need." Advances in Neural Information Processing Systems (2017).

Model evaluation

Evaluate performance on **test set** never seen during training

- Apply model on test set & compute metrics
- Test time augmentation
- **Manual** inspection of predictions
- Check how well the model handles augmented data
- Predict **out-of-distribution** samples

Empirical confidence interval with **bootstrap**

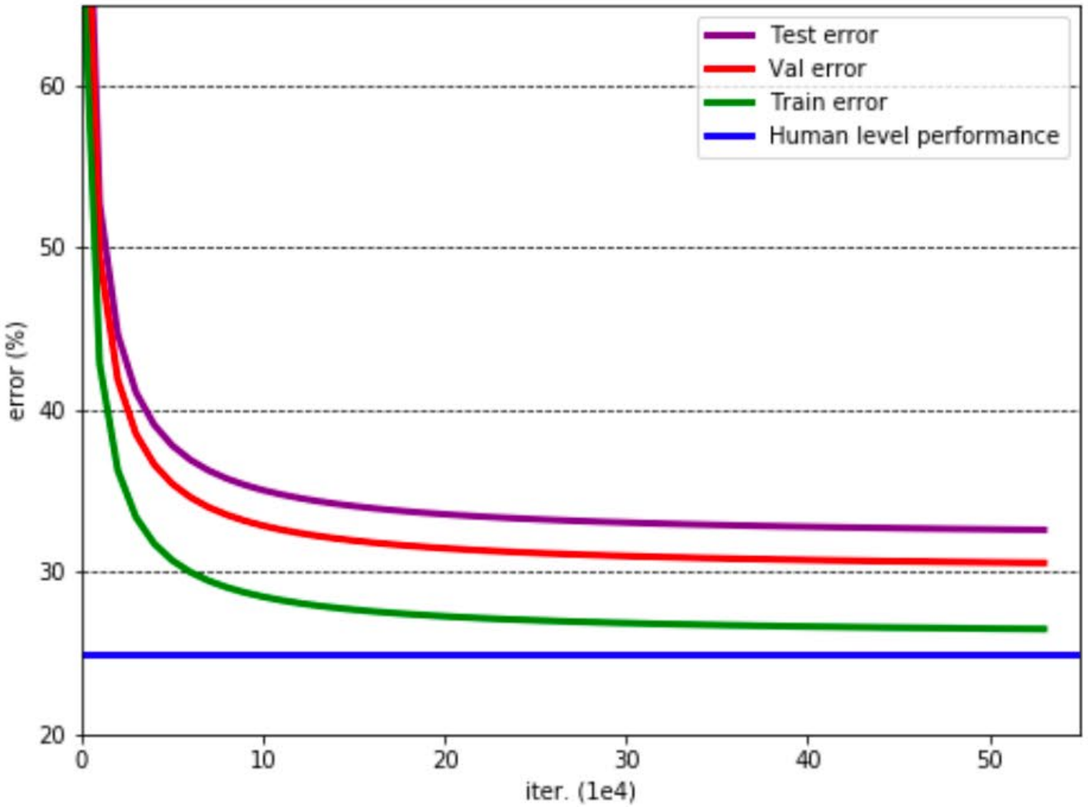
- Resample test set N times with replacement
- Evaluate test performance and compute confidence interval

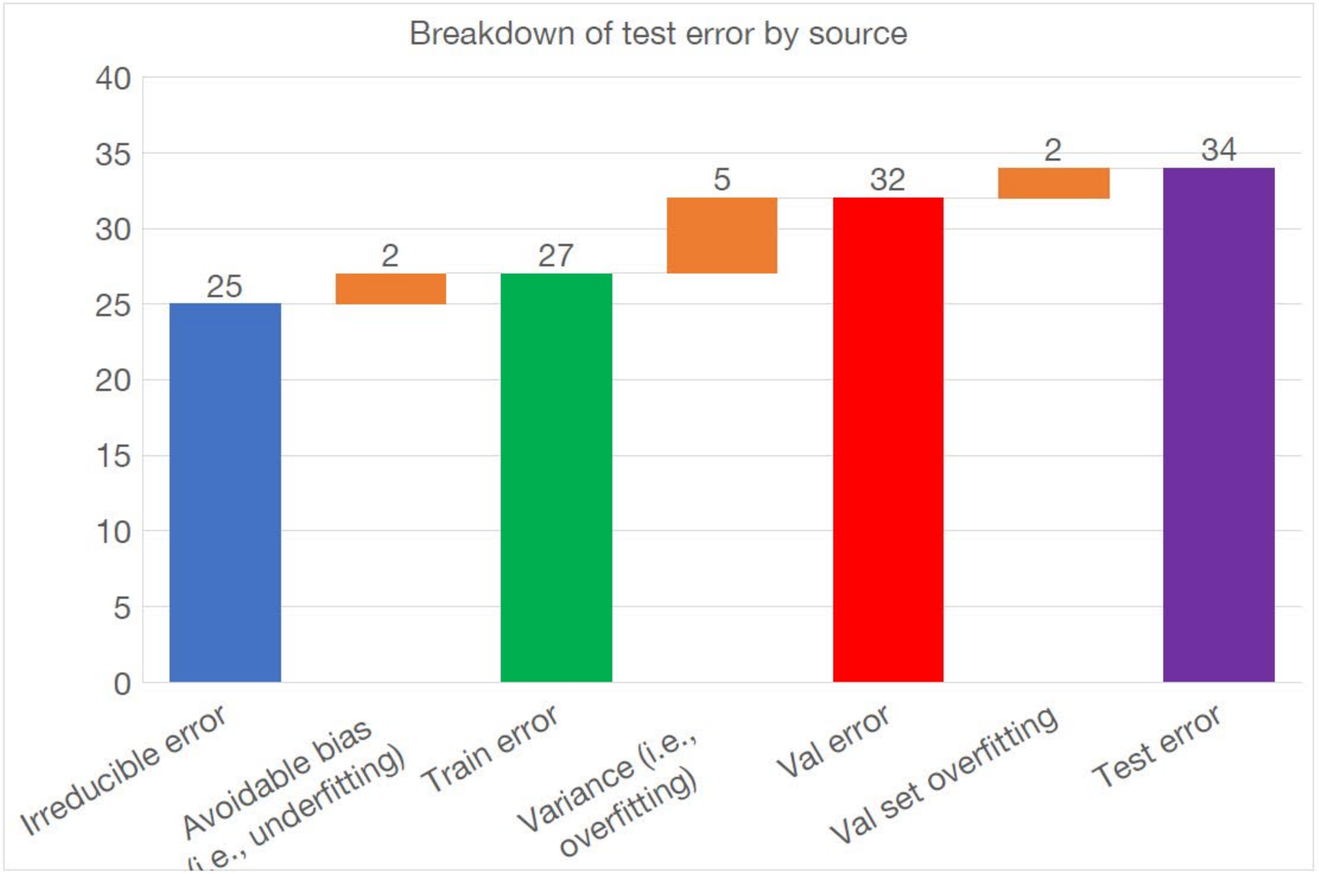
Evaluate model **uncertainty** with Monte Carlo dropout

- During training, dropout randomly drops neurons to improve model generalizability
- Use dropout at test time, predict test samples N times and evaluate entropy of the mean class probabilities

Error breakdown

Josh Tobin, 2019





Improve model performance – Strategies

Reassess ml & application goals

- Are ML objectives **well-aligned** with the actual modeling goals?
- Are the metrics relevant?
- Perform a **qualitative analysis**

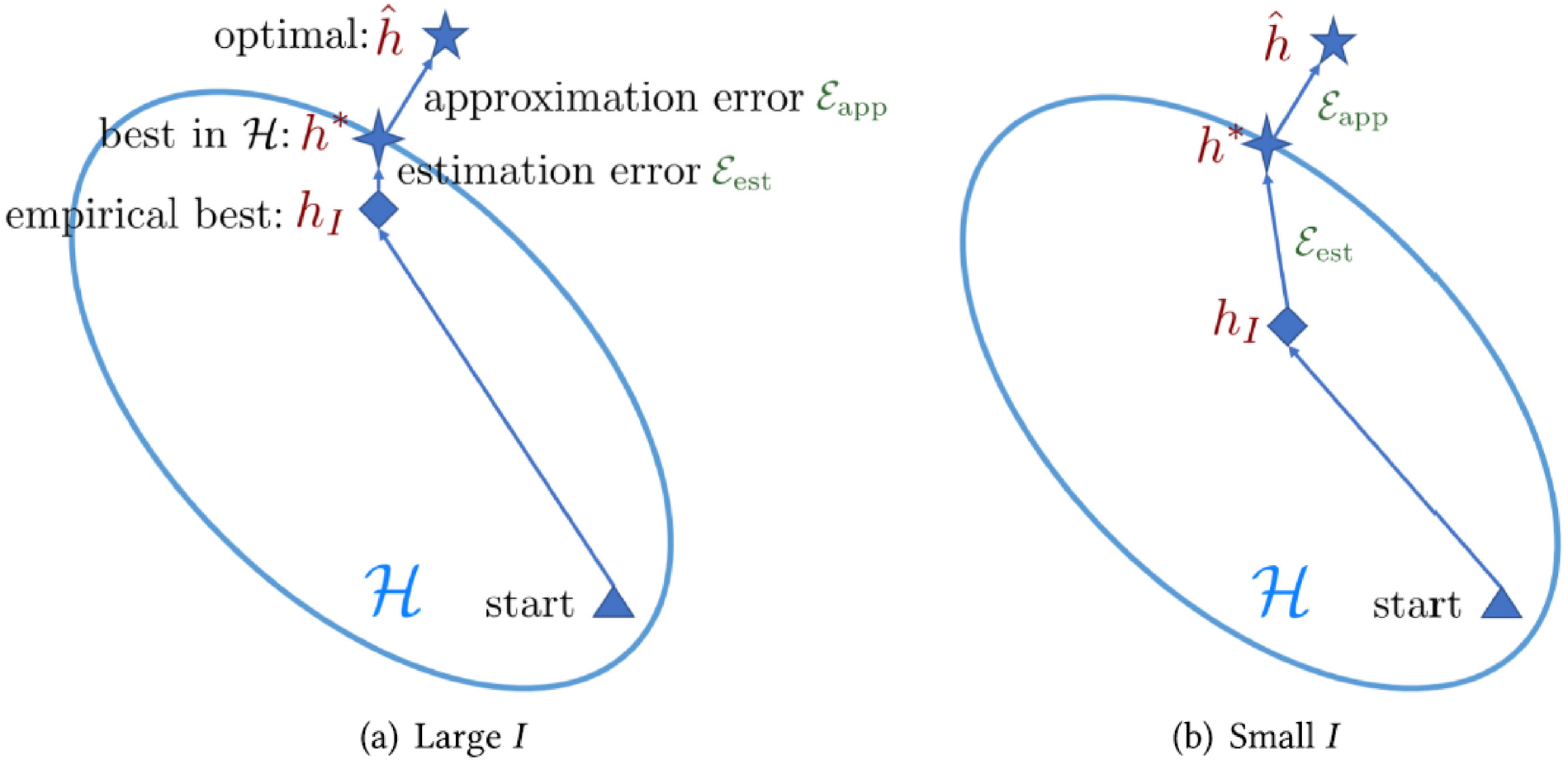
Leverage **expert knowledge**

- Restrict problem complexity
- Interpret model strengths & weaknesses
- Analyze model features

Engineering approach

- Analyze existing approaches and results
- Model tuning (hyperparameters, loss, etc)
- (Model ensembling)

Data scarcity settings

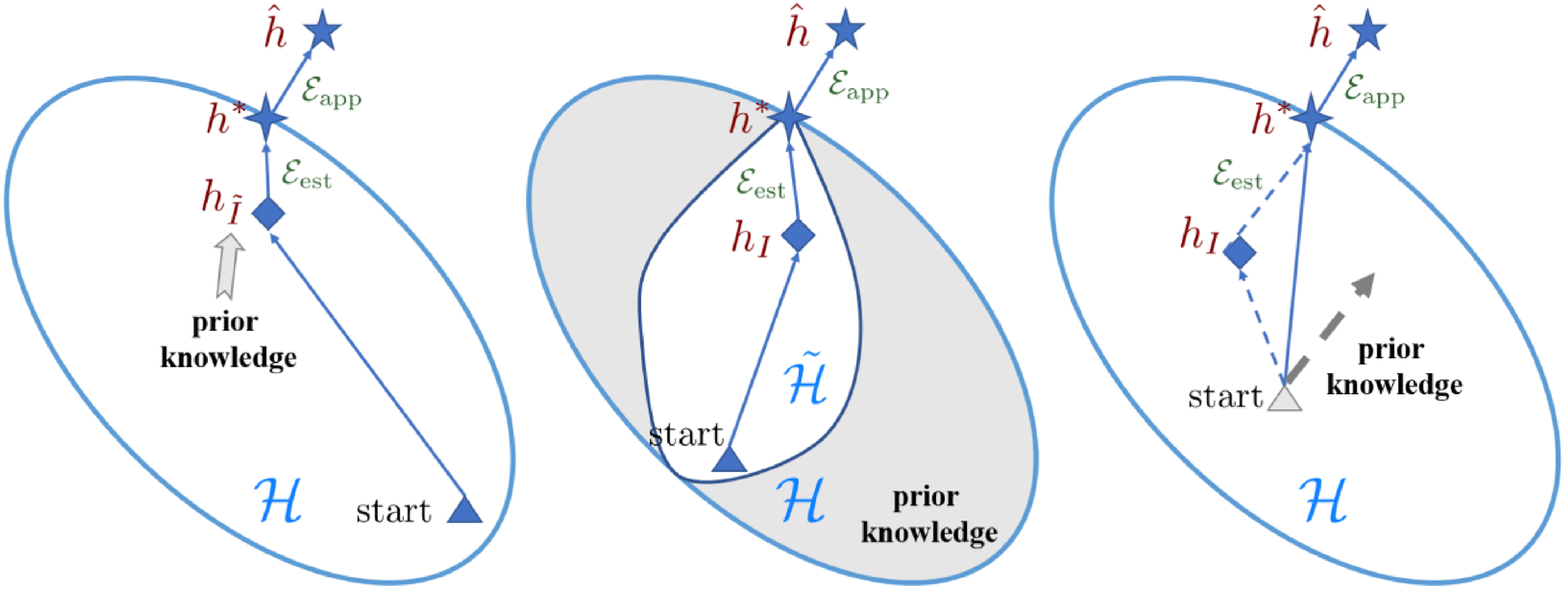


(a) Large I (b) Small I

Fig. 1. Comparison of learning with sufficient and few training samples.

Wang et al. 2020, "Generalizing from a Few Examples: A Survey on Few-shot Learning"

Mitigating data scarcity using prior knowledge

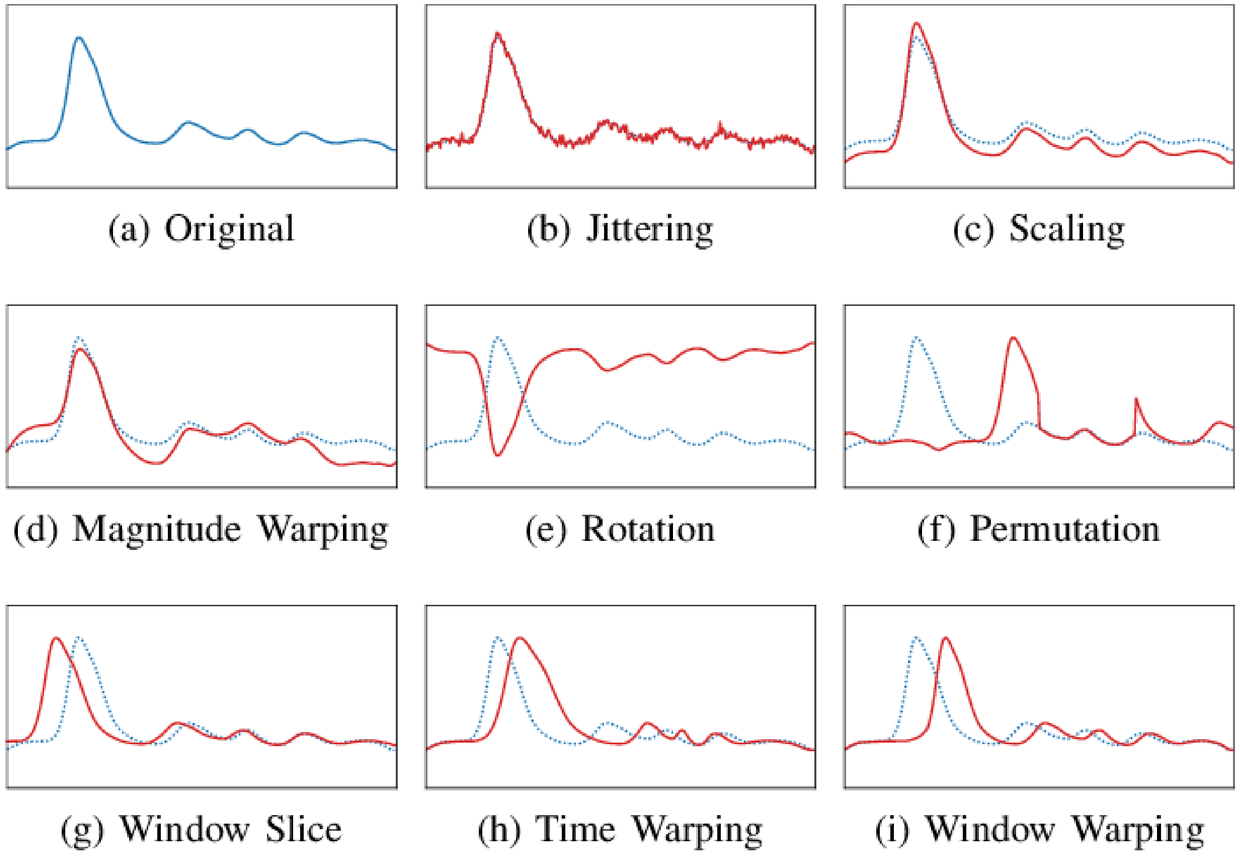


(a) Data. (b) Model. (c) Algorithm.

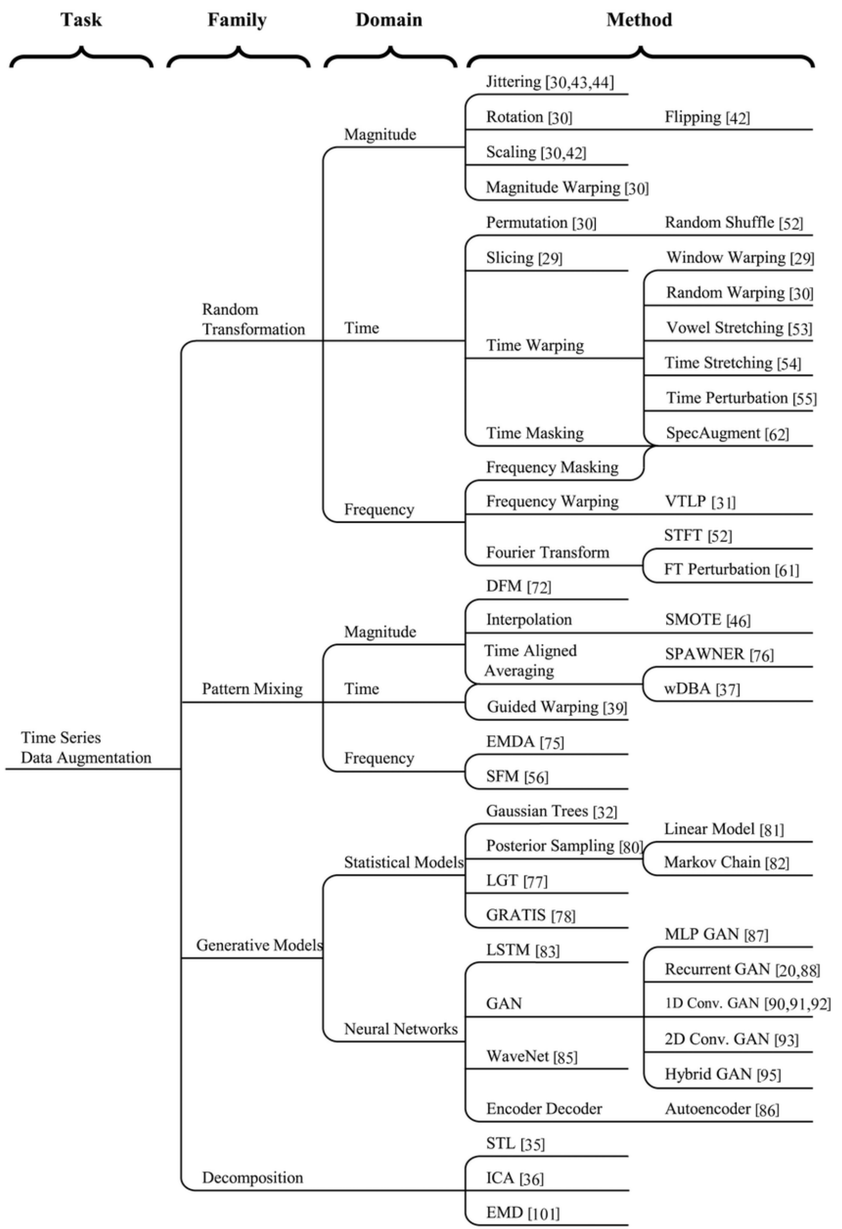
Fig. 2. Different perspectives on how FSL methods solve the few-shot problem.

Wang et al. 2020, "Generalizing from a Few Examples: A Survey on Few-shot Learning"

Data augmentation



Iwana et al. "Time series data augmentation for neural networks by time warping with a discriminative teacher." *2020 25th International Conference on Pattern Recognition (ICPR)*



Iwana et al. "An empirical survey of data augmentation for time series classification with neural networks." Plos one 16.7 (2021)

Practical considerations

Normalize per series or per channel to avoid scale bias.

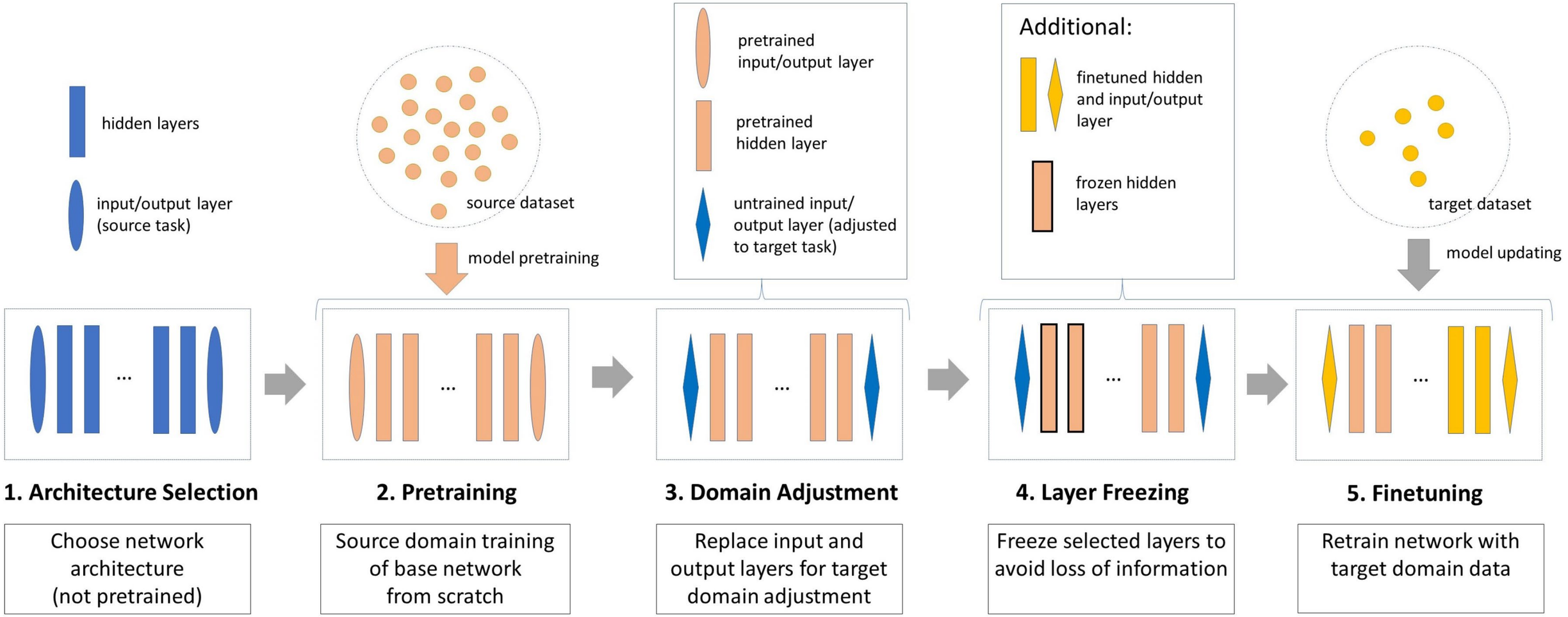
Align inputs and targets carefully to avoid **temporal leakage**.

Handle irregular sampling with **interpolation** or **masking** (placeholders for missing samples).

Multivariate series require consistent **scaling** and feature **grouping**.

Covariates with future information must be treated cautiously.

Transfer Learning Process



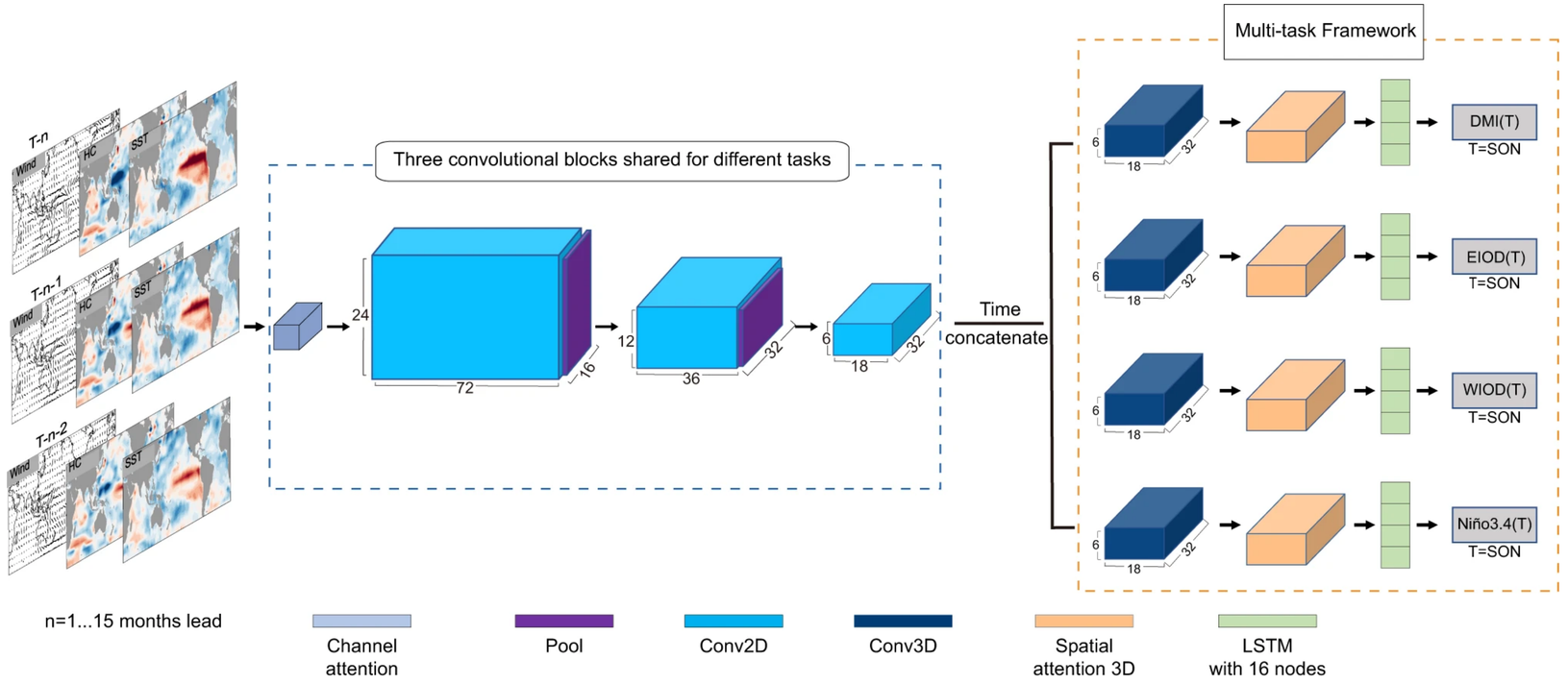
Gauge et al. "Industrial transfer learning for multivariate time series segmentation: A case study on hydraulic pump testing cycles." Sensors (2023).

Multi-task learning

Train model on multiple **related tasks** simultaneously.

Aggregate task losses

- **Learn** weighting scheme



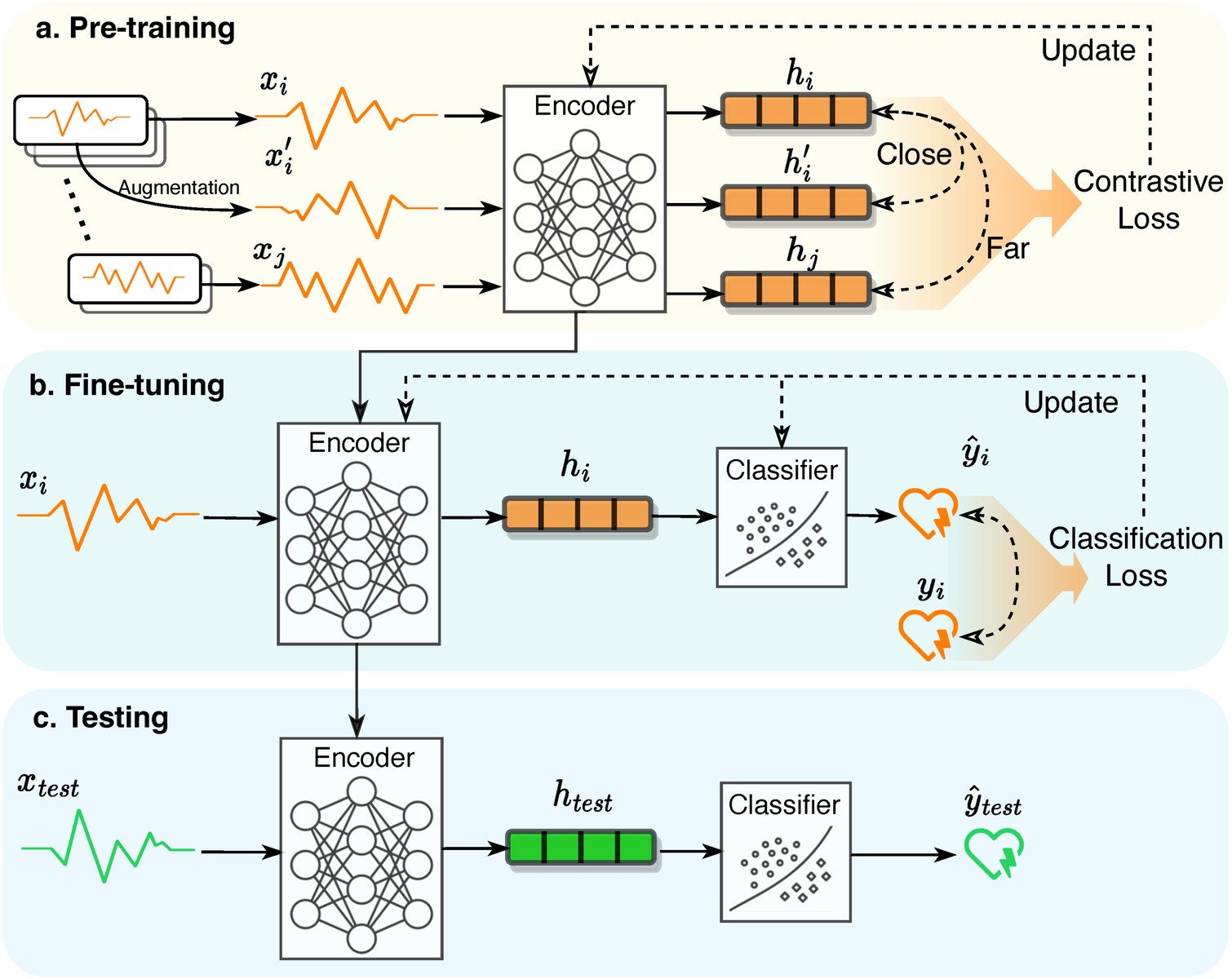
Ling, Fenghua, et al. "Multi-task machine learning improves multi-seasonal prediction of the Indian Ocean Dipole." Nature Communications 13.1 (2022): 7681.

Share feature representation to improve generalization (regularization effect)

- Hard parameters sharing
- Soft parameters sharing (enforce similarity)

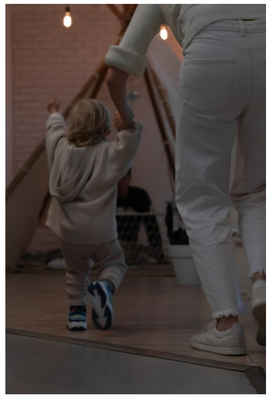
Improve **data efficiency** and reduce computation requirements at inference.

Self-supervised learning



Liu, Ziyu, et al. "Self-supervised contrastive learning for medical time series: A systematic review." Sensors 23.9 (2023): 4221.

DL In practice – Starting small



Start with **minimal complexity**

- Small input size
- Small, validated training set (use benchmark datasets such as CIFAR
- Normalize input
- No data augmentation
- Simple baseline model
- Adam optimizer
- Simple training objective
- Minimal codebase

DL In practice – Debugging

Does it **run**?

- Programming bugs ☺
- Wrong tensor shapes
- Wrong tensor types
- Out of GPU memory

Does it **learn**?

- Can you overfit the training data?
- Incorrect loss (e.g. activated logits instead of logits, numerical instabilities, use tf.math operations)
- Is the learning rate too high? Too low?
- Has the model sufficient capacity?
- If you used a benchmark dataset, compare with published results

DL In practice – gradual increase in complexity

1. Set actual training objective

2. Add data augmentation

3. Increase model complexity and regularization

4. Tune hyper parameters: manual/grid/random/Bayesian search
   - Check "A disciplined approach to neural network hyper-parameters", Leslie 2018

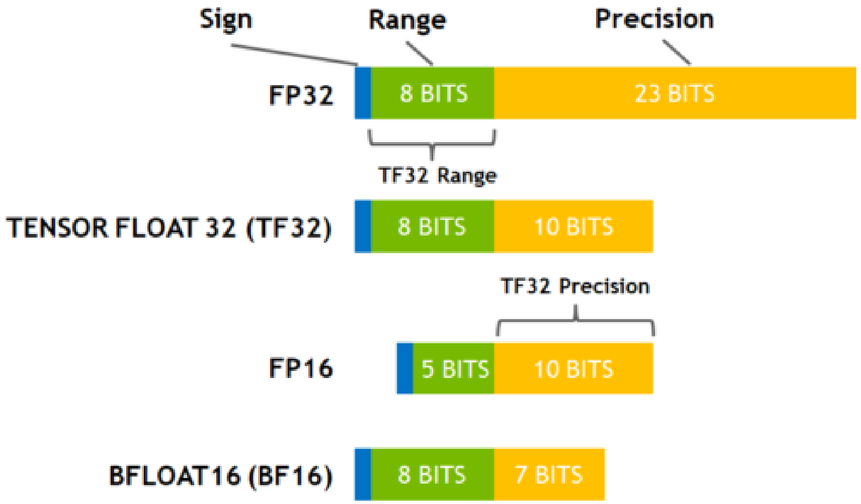
5. Finally, use training callbacks: reduce lr on plateau, early stopping, etc

Josh Tobin, 2019

Hyperparameter	Approximate sensitivity
Learning rate	High
Optimizer choice	Low
Other optimizer params (e.g., Adam beta1)	Low
Batch size	Low
Weight initialization	Medium
Loss function	High
Model depth	Medium
Layer size	High
Layer params (e.g., kernel size)	Medium
Weight of regularization	Medium
Nonlinearity	Low

Doing more with less: optimizing GPU memory usage

nvidia technical blog



Tune input size & batch size

Optimizer choice: Adam maintains two states per model parameter

- SGD, Adafactor, quantized Adam

Mixed precision training: computational speedup + enables larger batch size

- Activations are stored with half precision
- Model weights and gradients are stored in both half and full precision

Gradient accumulation: multiple forward/backward passes with smaller batch size before optimization step

Gradient checkpointing: selectively recompute activations during backward pass rather than storing them

- *"Fit 10x larger neural nets into memory at the cost of an additional 20% computation time."*, check article.

Exercise

Review the following resources

- Kaggle tutorial Intro to Deep Learning.
- Understanding LSTM Networks blog post from Christopher Olah.
- Troubleshooting Deep Neural Networks by Josh Tobin
- A Recipe for Training Neural Networks by Andrej Karpathy
- Practical Advice for Building Deep Neural Networks by Matt & Daniel

Model real-world time series.

- Train time series DL models.
- Make data stationary and retrain models.
- Compare with statistical and ML time series models.